

SILAB[®]

Version 7.1
Tutorial Szenariendesign

INHALT

1	EINLEITUNG	3
2	AUFGABENSTELLUNG	3
3	VORBEREITUNG.....	4
4	SZENARIO DESIGN.....	4
4.1	Schritt 1: Verlauf der Straße in der Ebene	4
4.2	Schritt 2: Höhenprofil	8
4.3	Schritt 3: Querschnittsprofil	11
4.4	Schritt 4: Landschaft	16
4.5	Schritt 5: Ausbau des Straßennetzes	21
4.6	Schritt 6: Vorausfahrendes Fahrzeug	24
4.7	Schritt 7: Mitverkehr	28
4.8	Schritt 8: Gegenverkehr	31
4.9	Schritt 9: Datenaufzeichnung (DPU-Erweiterungen)	33
4.10	Schritt 10: Test und Auswertung	36
4.10.1	Segmentierung der Daten	38
4.10.2	Berechnung abgeleiteter Maße	38
4.10.3	Berechnung von Kennzahlen und Interpretation der Ergebnisse	39
4.11	Schritt 11: Auffahrt aus Parkplatz (SILABAEEdit)	40
5	WIE GEHT ES WEITER?	50
6	REFERENZEN	52

1 EINLEITUNG

In diesem Tutorial lernen Sie die wichtigsten Schritte kennen, um mit SILAB eigene Szenarien zu entwerfen. Am Ende des Tutorials werden Sie wissen,

- wie man Straßennetze entwirft,
- wie man diese Straßennetze mit umgebende Landschaft ausgestaltet,
- wie simulierte Verkehrsteilnehmer eingesetzt werden, um bestimmte Verkehrssituationen herzustellen und
- wie Daten aus der Simulation aufgezeichnet und ausgewertet werden.

In Abschnitt 4 werden Sie in 11 Schritten ein einsatzfertiges und gebräuchliches Szenario entwickeln, mit dem Sie den Einfluss von Bedienkonzepten oder Nebenaufgaben auf die Fahrleistung beurteilen können. Die Aufgabenstellung ist im nächsten Abschnitt (2) detaillierter beschrieben. Abschnitt 0 beschreibt die notwendigen Vorbereitungen für dieses Tutorial. Natürlich können in diesem Tutorial nicht alle Features und Möglichkeiten der Szenariengestaltung mit SILAB behandelt werden. Der letzte Abschnitt 5 enthält daher Hinweise, wie Sie die Arbeit mit SILAB über das Tutorial hinaus fortsetzen können.

2 AUFGABENSTELLUNG

Das Szenario, das wir im Lauf dieses Tutorials entwickeln werden, ist Teil eines standardisierten Testprotokolls zur Messung der fahrerseitigen Ablenkung bedingt durch die Bedienung von Nebenaufgaben während des Fahrens.

Das Testprotokoll ist angelehnt an das von der Alliance of Automobile Manufacturers (AAM, 2006) vorgeschlagene Vorgehen. Generell soll hierbei eine standardisierte Fahraufgabe geschaffen werden, innerhalb welcher etwaige Beeinträchtigungen in Fahrzeugführung (d.h. longitudinale und laterale Fahrzeugführung) oder Aufmerksamkeitsausrichtung (d.h. Blickaktivität, z.B. mittlere oder maximale Blickabwendungszeiten) reproduzierbar und unter vergleichbaren Bedingungen erfasst werden kann. Beispielsweise können somit die Beeinträchtigungen durch die Bedienung neuartiger Anzeige- und Bedienelemente im Vergleich zu einer oder mehreren Referenzaufgaben (z.B. manuelles Einstellen des Radios) bestimmt werden¹. Die Prüfungsumgebung soll hierbei folgende Eigenschaften haben (siehe AAM, 2006)

- Umgebung: geringe bis moderate Verkehrsdichte, gute Licht- und Witterungsbedingungen
- Streckennetz: geteilte, mehrstreifige Straße mit Geschwindigkeitslimit von 45mph oder darunter
- Vorderfahrzeug: vorausfahrendes Fahrzeug mit Geschwindigkeits-Variabilität

¹ Das genaue Vorgehen bei der Durchführung des Test-Protokolls, beispielsweise Eigenschaften der Stichprobe, Instruktion oder Ablauf der Untersuchung ist in AAM (2006) zu finden.

- Datenaufzeichnung: Aufzeichnung von Maßen der lateralen Fahrzeugkontrolle (z.B. Spurposition) und longitudinalen Kontrolle (z.B. Folgeabstand)

3 VORBEREITUNG

Im Unterordner „Projects\SILABTutorial\SzenarioDesign“ haben wir einige Dateien vorbereitet, die Sie im Laufe dieses Tutorials modifizieren werden. Damit diese Originaldateien nicht verloren gehen, erstellen Sie bitte eine Sicherungskopie dieses Ordners.

Bei den vorbereiteten Dateien handelt es sich hauptsächlich um Text-Dateien. Sie benötigen deshalb einen Texteditor, um sie zu bearbeiten. Die Beschreibungen in diesem Tutorial beziehen sich auf den Texteditor Geany, der unter <https://www.geany.org/> frei erhältlich ist. Sie können auch irgendeinen anderen Texteditor verwenden, der die Möglichkeit bietet, externe Kommandos auszuführen (z.B. Notepad++, UltraEdit). Wenn Sie keinen Zugriff auf einen solchen Editor haben, dann verwenden Sie zur Not und mit Einschränkungen den Editor aus dem Zubehör von Windows.

Im Lieferumfang von SILAB ist ein Hilfsprogramm enthalten, das bei der Szenarienerstellung sehr hilfreich ist (mehr dazu später). Integrieren Sie dieses in Geany wie in der Datei „Handbuch SILAB Essential“ im Kapitel 9.4.1 „Installation und Einrichten des Texteditors Geany“ beschrieben.

4 SZENARIO DESIGN

4.1 Schritt 1: Verlauf der Straße in der Ebene

Öffnen Sie die Skript-Datei „tutorial_step_01.cfg“ in Ihrem Texteditor und überfliegen Sie den Inhalt. Sie sehen an vielen Stellen Zeilen der Form

```
### irgendein Text
```

Zu jeder dieser Zeilen finden Sie weiter unten eine entsprechende Zeile der Form

```
### /irgendein Text
```

Damit haben wir wichtige Abschnitte markiert, die jeweils einen bestimmten Typ von Informationen enthalten. Da SILAB das Zeichen „#“ als Beginn eines Kommentars interpretiert und alles, was in der Zeile nach diesem Zeichen folgt, ignoriert, dienen diese Markierungen rein dazu, die Informationen in der Datei zu strukturieren.

Wenn Sie nur auf die so markierten Abschnitte achten, dann ist die Datei wie folgt aufgebaut:

```
### simulator configuration
### /simulator configuration

### component configuration
### /component configuration

### scenario definition

    ### cross section profile
    ### /cross section profile

    ### scenario module
        ### course
            ### planar road geometry
            ### /planar road geometry
        ### /course

        ### road network
        ### /road network
    ### /scenario module

    ### map
    ### /map

### /scenario definition
```

Bitte beachten Sie:

- Der Beginn eines Abschnitts ist mit `### Abschnittsname` markiert, er endet mit der Markierung `### /Abschnittsname`.
- Abschnitte können Unterabschnitte enthalten. Sie erkennen das daran, dass Unterabschnitte gegenüber ihrem übergeordneten Abschnitt eingerückt sind. Beispielsweise sind die Abschnitte `cross section profile`, `scenario module` und `map` Unterabschnitte von `scenario definition`.

In diesem Schritt des Tutorials konzentrieren wir uns auf den Verlauf der Straße in der Ebene. Er wird im Abschnitt `planar road geometry` festgelegt. Suchen Sie diesen Abschnitt in der Datei. Er sieht wie folgt aus:

```
### planar road geometry
Straight(1000);
### /planar road geometry
```

Die Definition `Straight(1000);` innerhalb des Abschnitts erzeugt ein gerades Stück Straße mit einer Länge von 1000 m.

Das wollen wir uns in der Simulation anschauen:

1. Starten Sie hierzu das Programm „SILAB.exe“. Wenn Sie noch keine Verknüpfung zu diesem Programm auf Ihrem Desktop haben, dann finden Sie es im Verzeichnis „bin“ Ihres SILAB-Installationsordners auf der Festplatte.
2. Wählen Sie *Datei* → *Öffnen* und stellen Sie in dem Dialog, der jetzt erscheint, den Datei Typ auf „Konfigurationsdateien (*.cfg)“.

3. Öffnen Sie unsere Datei „tutorial_step_01.cfg“. SILAB verarbeitet jetzt die darin enthaltenen Informationen, was einige Momente dauern kann.
4. Sobald das Toolbar-Icon für den Befehl *Launch* die Farbe von grau auf gelb wechselt (☀️), können Sie die Simulation starten. Klicken Sie auf das *Launch*-Icon in der Toolbar.
5. Klicken Sie auf das Toolbar-Icon für den Befehl *Start*, sobald es blau ist (▶️). Sie sehen jetzt in einem zusätzlichen Fenster folgendes (oder ein sehr ähnliches) Bild:



6. Sie können folgendermaßen auf der Straße fahren:
 - Wenn Sie gerade an Ihrem Fahrsimulator arbeiten, dann verwenden Sie einfach Gas, Bremse und Lenkung Ihres Mockups, um das Fahrzeug zu steuern.
 - Wenn Sie an einem einzelnen Rechner ohne Mockup arbeiten, dann verwenden Sie die Cursor-Tasten zur Steuerung des Fahrzeugs.
7. Wenn Sie sich genug umgesehen haben, beenden Sie SILAB mit dem Befehl *Datei* → *Beenden*.

Wie Sie beim Fahren sicher bemerkt haben, ist die Straße noch ziemlich langweilig. Sie besteht nämlich genau aus dem 1 km langen Geradenstück, das in unserer Datei „tutorial_step1.cfg“ mit der Anweisung `Straight(1000);` definiert ist. Für unsere Aufgabenstellung benötigen wir einen etwas interessanteren Straßenverlauf, den wir jetzt entwerfen werden.

Wir möchten einen 5 km langen Autobahnabschnitt mit sich abwechselnden Kurven und Geraden. Neben der Anweisung `Straight(<Länge in m>)` für die Geradenstücke gibt es die Anweisung `Bend(<Länge in m>, <Radius in m>)` für Kurven. (Die Spitzen Klammern dienen im Text nur als Platzhalter für die tatsächlichen Werte.) Legen Sie mit diesen Anweisungen den Straßenverlauf wie folgt fest:

```
### planar road geometry
Straight(200);
Bend(1000, 2000);
Bend(1500, -3000);
Straight(100);
Bend(1000, -2500);
Straight(200);
Bend(1000, 2700);
### /planar road geometry
```

Bitte beachten Sie:

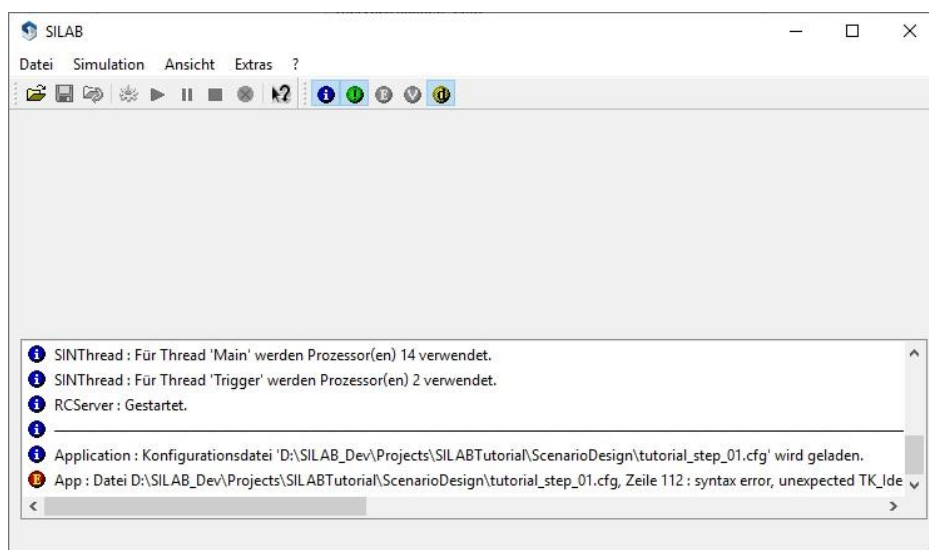
- Alle Längen der `Straight` und `Bend` Stücke summieren sich zu unserer gewünschten Gesamtlänge von 5 km.
- Die Stücke erscheinen bei der Fahrt genau in der Reihenfolge, in der sie Sie in der Datei definiert haben.
- Wir verwenden autobahntypische Kurvenradien ab 2 km.
- Ein negativer Kurvenradius entspricht einer Linkskurve, ein positiver entspricht einer Rechtskurve.

Starten Sie SILAB und durchfahren Sie den jetzt interessanteren Straßenverlauf wie oben beschrieben. Sie können gerne auch mit den Parametern für Länge und Kurvenradius „spielen“ oder ganz eigene Straßenverläufe definieren.

Noch ein Wort zu Fehlern beim Arbeiten mit der Skript-Datei:

Damit SILAB die Informationen, die Sie in der Skript-Datei eingegeben haben, verarbeiten kann, gibt es bestimmte Regeln, wie diese eingegeben werden müssen. Diese Regeln werden auch als „Syntax“ des Skripts bezeichnet. Beispielsweise sehen Sie in unserem Straßenverlauf, dass jede Definition eines `Straight` und `Bend` Stücks mit einem Strichpunkt abgeschlossen werden muss.

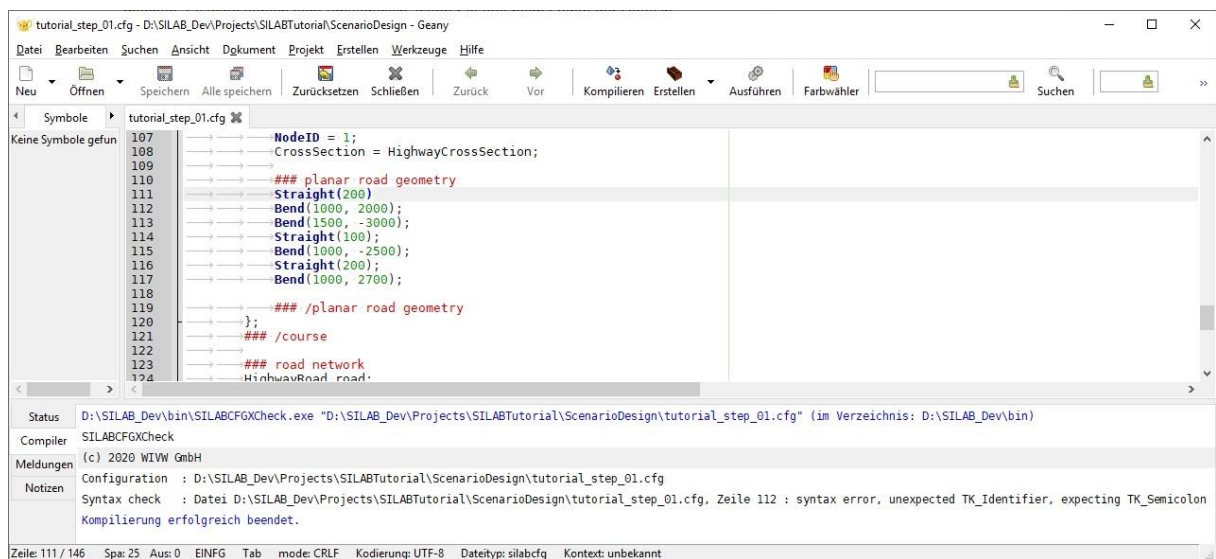
Wenn Sie das vergessen und die Skript-Datei zum Fahren in SILAB öffnen, dann erscheint eine Fehlermeldung und SILAB bricht die Verarbeitung ab. Das machen wir jetzt einmal absichtlich, indem Sie bitte den abschließenden Strichpunkt nach dem ersten `Straight(200)` Stück entfernen und die „fehlerhafte“ Datei in SILAB laden. Im Meldungsfenster im unteren Bereich von SILAB sehen Sie jetzt folgendes:



In der letzten Zeile weist SILAB Sie auf den Syntax-Fehler hin. Sie erhalten folgende Angaben:

- Der Name der Skript-Datei, in der sich der Fehler befindet.
- Die Zeile, in der SILAB bemerkt hat, dass die Skript-Datei einen Fehler enthält. Bitte beachten Sie, dass diese Angabe nicht genau der Zeile mit dem Fehler entsprechen muss. Meistens werden eine oder ein paar Zeilen nach der Stelle gemeldet. In unserem Beispiel hat SILAB das Fehlen des Strichpunkts in der Zeile unmittelbar nach der fehlerhaften `straight` Anweisung bemerkt.
- Die Art des Fehlers und einige nähere Angaben dazu. In unserem Fall also, dass es sich um einen „syntax error“ handelt und dass SILAB einen Strichpunkt erwartet hätte („expecting TK_Semicolon“).

Beim Bearbeiten von Skript-Dateien ist es nur normal, dass Sie versehentlich Fehler machen. Es wäre jedoch sehr umständlich, wenn Sie erst beim Laden der Datei in SILAB darauf hingewiesen werden würden. Deswegen haben wir in Abschnitt 0 ein Tool in Ihren Texteditor integriert, das die Überprüfung des Skripts schon im Texteditor vornimmt. Wenn Sie Geany verwenden, dann wählen Sie im Menü *Erstellen* → *Syntaxcheck 7.0*. Im „Output“ Fenster von Geany erhalten Sie jetzt die Fehlermeldung:



Eine gute Angewohnheit ist, vor jedem Laden einer Skript-Datei einen solchen Check im Texteditor durchzuführen und einen eventuellen Fehler auszubessern. Diese Prozedur ist solange zu wiederholen, bis die Ausgabe „Syntax-Check: OK.“ erscheint.

4.2 Schritt 2: Höhenprofil

Neben einem interessanten Verlauf der Straße in der Ebene entwerfen wir in diesem Schritt auch ein Höhenprofil, dem die Straße folgt. Öffnen Sie hierzu die Datei „tutorial_step_02.cfg“ in Ihrem Texteditor. Wie Sie im Abschnitt `### planar road geometry` sehen, haben wir den Straßenverlauf aus dem letzten Kapitel in diese Datei übernommen. Wir konzentrieren uns in diesem Schritt auf den Abschnitt, der mit


```
### height profile
### /height profile
```

markiert ist. Er befindet sich auf derselben Unterabschnittsebene wie `### planar road geometry`. Der Grund hierfür ist, dass das Höhenprofil einer Straße unabhängig von ihrem Verlauf in der Ebene ist.

Fügen Sie folgende Definitionen ein:

```
### height profile
HeightProfile =
{
    ( 0.0, 500.0, 4000),
    ( 0.5, 400.0, 4000),
    ( 0.0, 600.0, 4000),
    (-0.8, 500.0, 4000),
    ( 1.0, 800.0, 4000),
    ( 0.0, 600.0, 4000),
    (-0.6, 400.0, 4000),
    ( 0.0, 300.0, 4000),
    ( 0.4, 700.0, 4000),
    ( 0.0, 200.0, 4000)
};
### /height profile
```

Bitte beachten Sie:

- Das Höhenprofil besteht aus einer Abfolge von Segmenten, die entweder eben sind oder eine Steigung bzw. ein Gefälle haben.
- Jedes dieser Segmente hat eine bestimmte Länge.
- In der Definition entspricht jedes Element mit runden Klammern einem solchen Segment. Die Informationen in den Klammern bedeuten folgendes:

(<Steigung in %>, <Länge des Segments in m>, <Ausrundungshalbmesser>)

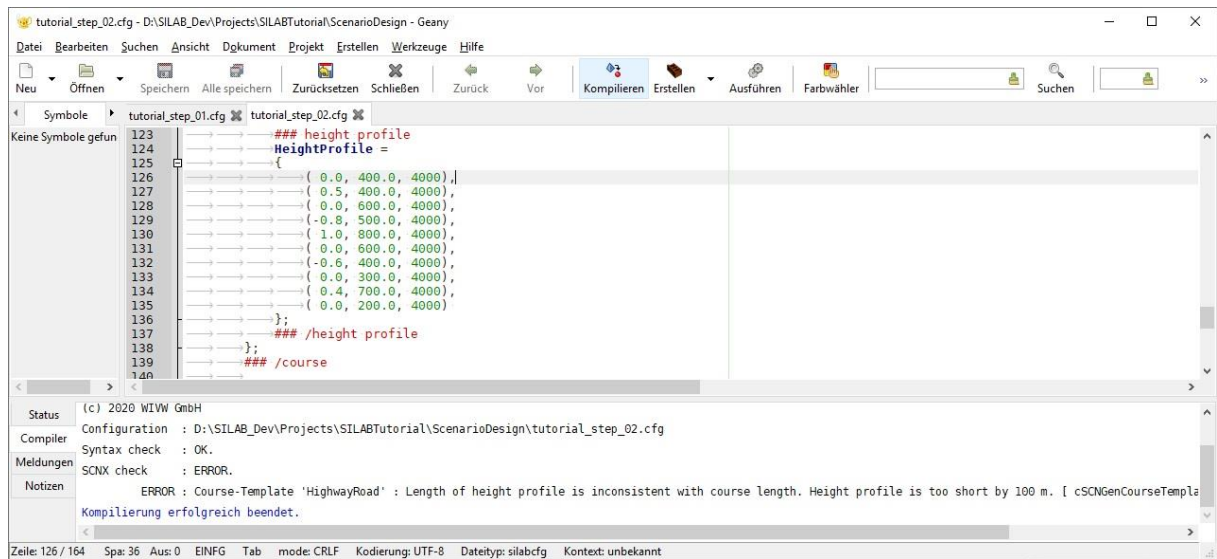
Beispielsweise hat das zweite Segment eine Steigung von 0.5%, ist 400 m lang und hat einen Ausrundungshalbmesser von 4000 m.

- Die Steigungen werden in % angegeben. Beispielsweise bedeutet eine Steigung von 0.5%, dass die Straßenhöhe nach 100 m (entlang der Straße) um 0.5 m ansteigt. Ein positiver Wert entspricht einem Anstieg, ein negativer Wert einem Gefälle. Eine Steigung von 0% entspricht einem flachen Segment.
- Würde SILAB einfach nur die einzelnen Segmente aneinanderfügen, dann würden im Straßenverlauf Knicke entstehen, nämlich immer an den Stellen, an denen sich die Steigung ändert. Im Straßenbau werden daher sog. Ausrundungen an diesen Stellen vorgeschrieben. Die Form und Größe dieser Ausrundungen, die SILAB automatisch einfügt, können mit dem Parameter <Ausrundungshalbmesser> beeinflusst werden. In den allermeisten Fällen können Sie hier einfach den Wert 4000 eintragen.
- Das erste und das letzte Segment eines Höhenprofils muss immer die Steigung 0% haben. Den Grund hierfür lernen Sie später kennen (s. Abschnitt 4.5).
- Die Gesamtlänge aller Segmente des Höhenprofils muss der Gesamtlänge des Straßenverlaufs in der Ebene entsprechen. In unserem Beispiel sind dies 5km.

Laden Sie die Datei „tutorial_step_02.cfg“ in SILAB und durchfahren Sie unsere Autobahn. (Die Vorgehensweise ist im vorhergehenden Abschnitt beschrieben.) Achten Sie dabei auf die automatisch generierten Ausrundungen zwischen den Segmenten und beobachten Sie, wie SILAB die Landschaftshöhen links und rechts der Straße automatisch „glatt“ mit dem Höhenprofil der Straße mitführt. Experimentieren Sie auch gerne mit anderen Höhenprofilen.

Noch ein Wort zu Fehlern beim Arbeiten mit der Skript-Datei (Fortsetzung):

Neben den Syntaxfehlern, auf die im vorhergehenden Kapitel an dieser Stelle eingegangen wurde, gibt es auch Fehler, die mit dem logischen Aufbau der Streckendefinition zu tun haben. Ein Beispiel für so einen Fehler ist, wenn die Gesamtlänge des Höhenprofils nicht mit der Länge des ebenen Verlaufs der Straße übereinstimmt. Wir machen das einmal absichtlich. Verkürzen Sie bitte ein Segment unseres Höhenplans (z.B. das erste) um 100m. Rufen Sie in Ihrem Editor das Tool *Syntaxcheck 7.0* auf. Sie bekommen folgende Meldung:



SILAB teilt Ihnen den Fehler mit und informiert Sie auch, wie groß der Längenunterschied ist („Höhenprofil ist um 100m zu kurz.“).

4.3 Schritt 3: Querschnittsprofil

In den beiden letzten Abschnitten haben wir uns mit dem Straßenverlauf beschäftigt, der sich aus zwei Teilen zusammensetzt: Dem Verlauf der Straße in der Ebene und ihrem Höhenverlauf. Es gibt noch einen weiteren Teil, mit dem eine Straße in SILAB (und auch in der Realität) beschrieben wird: Das Querschnittsprofil. Wie der Name schon sagt, enthält das Querschnittsprofil Informationen, die nichts mit dem Verlauf *entlang der Straße* zu tun haben, sondern mit Ihrem Aufbau im Querschnitt. Dazu gehört u.a.

- die Anzahl der Spuren,
- deren Breite und Höhe,
- deren Fahrtrichtung und Befahrbarkeit,
- deren Belag,
- Linienmarkierungen zwischen den Spuren,
- Bebauungen zwischen bzw. neben den Spuren (z.B. Leuchtpfosten, Leitplanken)
- Bebauungen auf den Spuren (z.B. Hecke auf einem Begrenzungstreifen in der Mitte einer Autobahn),
- der Übergang zwischen linker und rechter Spur und Landschaft, die sog. Schultern der Straße.

Das sind sehr viele Informationen und deswegen ist die Definition eines Querschnittsprofils auch etwas umfangreicher. Öffnen Sie die Datei „tutorial_step_03.cfg“ in Ihrem Editor und suchen Sie den Abschnitt, der mit `### cross section profile` beginnt. In diesem Abschnitt ist das Querschnittsprofil für unsere Autobahn definiert. Er sieht so aus:

```

### cross section profile
CrossSection HighwayCrossSection
{
    ve = 150.0;
    Shoulder = {1.0, 2.4, -0.7, Shoulder1};
    LaneInfos =
    {
        (0, 2.5, 0.0, Towards, Tar, 1.0, 0.0, { (None, 0.0) } ),
        (1, 3.75, 0.0, Towards, Tar, 1.0, 0.0, { (None, 0.0) } ),
        (2, 3.75, 0.0, Towards, Tar, 1.0, 0.0, { (None, 0.0) } ),
        (3, 3.0, 0.0, Towards, Grass1, 1.0, 0.0, { (None, 0.0) } ),
        (4, 3.75, 0.0, Onwards, Tar, 1.0, 0.0, { (None, 0.0) } ),
        (5, 3.75, 0.0, Onwards, Tar, 1.0, 0.0, { (None, 0.0) } ),
        (6, 2.5, 0.0, Onwards, Tar, 1.0, 0.0, { (None, 0.0) } )
    };
    LaneTransitions =
    {
        der Übersichtlichkeit halber wird hier nur
        ein Spurübergang aufgeführt (von Spur 1 nach 2)

        (1, 2,
            {
                (Line, 0.0, 0.15, Towards, 6.0, 12.0)
            }
        ),

    };
};
### /cross section profile

```

Die Definition beginnt mit dem Schlüsselwort `CrossSection`. Es folgt ein (möglichst aussagekräftiger) Name für das Querschnittsprofil, den Sie frei wählen dürfen. Unser Autobahnquerschnittsprofil heißt `HighwayCrossSection`. Es folgt ein Teil mit Angaben, die die ganze Straße betreffen. Danach werden mit `LaneInfos = { ... };` die einzelnen Spuren definiert. Jeder Eintrag in dieser Menge beschreibt eine Spur. Unsere Autobahn hat 7 Spuren, nämlich (von links nach rechts betrachtet):

- Spur 0: Standstreifen auf der Gegenfahrbahn
- Spur 1 und Spur 2: Fahrspuren auf der Gegenfahrbahn
- Spur 3: Begrünter Mittelstreifen
- Spur 4 und Spur 5: Fahrspuren in „Mit“ – Richtung
- Spur 6: Standstreifen

Die Spurnummern finden Sie auch als erste Angabe in jedem Eintrag. Die nächste Angabe legt die Breite einer jeden Spur fest. Die Standstreifen haben eine Breite von 2.5 m, die Fahrspuren von deutschen Autobahnen sind 3.75 m breit, der Mittelstreifen 3.0 m. Es folgen einige weitere Angaben für jede Fahrspur.

Nach dem Abschnitt `LaneInfos = { ... };` für die Spuren folgt ein Abschnitt `LaneTransitions = { ... };` mit Informationen zu den Übergängen zwischen Spuren. In obigem Beispiel ist (der Übersichtlichkeit halber nur) der Übergang von Spur 1 nach Spur 2 gezeigt. Das erkennen Sie an den beiden ersten Einträgen nach der Klammer. Es wird definiert, dass die beiden Spuren durch eine Linienmarkierung getrennt

sind und zwar – gemäß den deutschen Straßenbau-Vorschriften – mit einer Strichlänge von 6 m und 12 m Abstand zwischen den Strichen.

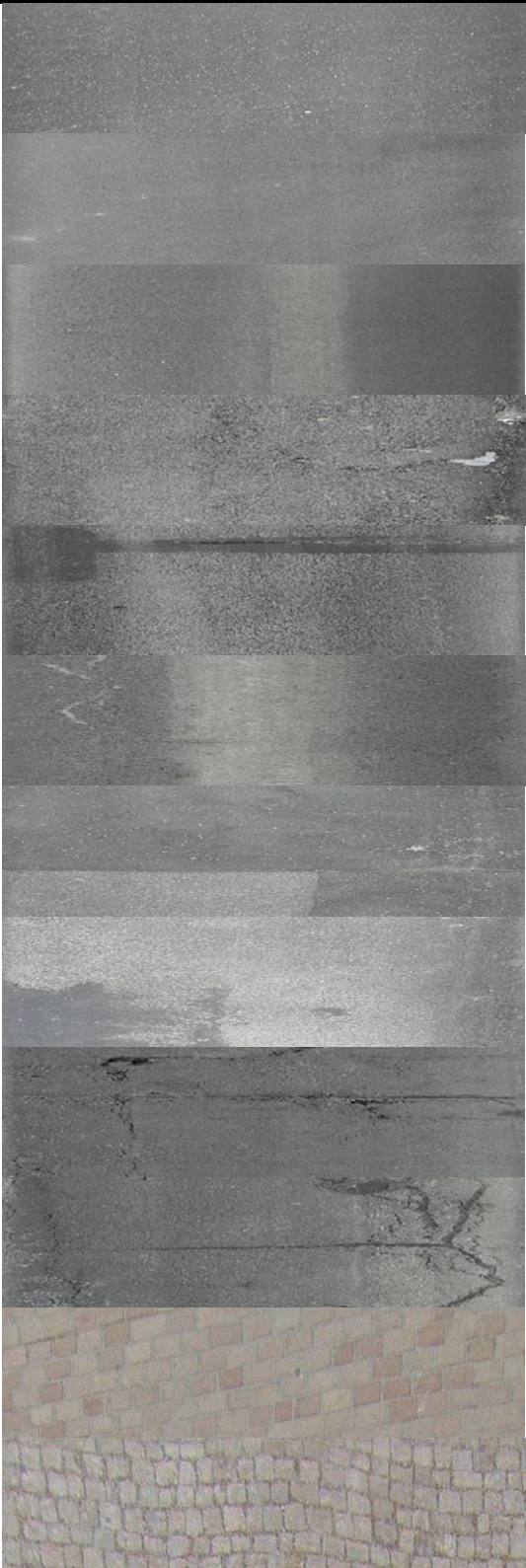
Für unsere Straße möchten wir gerne einen etwas realistischeren Straßenbelag. Fügen Sie hierzu gleich nach der Zeile `Shoulder = {1.0, 2.4, -0.7, Shoulder1}`; folgendes ein (rot markiert):

```
### cross section profile
CrossSection HighwayCrossSection
{
    ve = 150.0;
    Shoulder = {1.0, 2.4, -0.7, Shoulder1};
    Overlay = {TarNew1, 1.0};
    LaneInfos =
    {
        # usw.
```

Laden Sie die Datei in SILAB und durchfahren Sie die Strecke. Sie sieht jetzt wesentlich realistischer aus.



Der Straßenbelag `TarNew1`, den wir mit unserer Ergänzung als sog. Overlay auf die Straße gelegt haben, wurde aus Bildern von echten Straßen gewonnen. Experimentieren Sie mit folgenden weiteren Belägen, die im Lieferumfang von SILAB enthalten sind:

Name	Bild
TarNew1	
TarNew2	
TarNew3	
TarOld1	
TarOld2	
TarOld3	
TarOld4	
TarOld5	
TarBroken1	
TarBroken2	
Plaster1	
Plaster2	

Wie Sie gesehen haben, bieten die Querschnittsprofile von SILAB sehr viele Gestaltungsmöglichkeiten. Das geht bis hin zu regennassen Fahrbahnen und Tunnel. Allerdings sind Querschnittsprofile deswegen auch etwas aufwändiger zu definieren. In

der Regel werden Sie jedoch einen Querschnitt nicht jedes Mal von Grund auf neu definieren. Vielmehr werden Sie – wie in unserem Beispiel – einen bereits definierten Standardquerschnitt an Ihre Vorstellungen anpassen. Wir haben Ihnen in der Datei „CrossSectionProfiles.inc“ die wichtigsten zusammengestellt:

HighwayCrossSection	Diesen Querschnitt haben wir bisher in unserer Straßendefinition benutzt.
RuralMainRoadCrossSection	Querschnitt einer gut ausgebauten Landstraße (entspricht der deutschen Bundesstraße)
RuralRoadCrossSection	Querschnitt einer typischen Landstraße, die kleinere Ortschaften miteinander verbindet

Öffnen Sie die Datei „CrossSectionProfiles.inc“ in Ihrem Editor und überfliegen Sie die drei Abschnitte `CrossSection <Name> { ... };`

Die vordefinierten Querschnitte können Sie auf folgende Weise verwenden:

- Ändern Sie den Abschnitt `### cross section profile` unserer Datei „tutorial_step_03.cfg“ folgendermaßen ab:

```
### cross section profile
include CrossSectionProfiles.inc
### /cross section profile
```

Anstatt der langen Querschnittsdefinition wird die Datei „CrossSectionProfiles.inc“ mit dem `include` Befehl eingebunden. Beim Verarbeiten ersetzt SILAB einfach die Zeile `include CrossSectionProfiles.inc` mit dem Inhalt der eingebundenen Datei.

- Suchen Sie die mit `### cross section assignment` markierte Stelle:

```
### cross section assignment
CrossSection = HighwayCrossSection;
### /cross section assignment
```

```
### planar road geometry
```

(der Übersichtlichkeit halber nicht dargestellt)

```
### /planar road geometry
```

```
### height profile
```

(der Übersichtlichkeit halber nicht dargestellt)

```
### /height profile
```

Sie sehen vor sich alle drei Abschnitte, mit denen die Straße definiert wird: Querschnittsprofil, Verlauf in der Ebene und Höhenplan. Die Festlegung des Querschnitts ist dabei nur eine einzige Zeile der Form:

```
CrossSection = <Name des Querschnittsprofils>;
```

Verwenden Sie dabei als `<Name des Querschnittsprofils>` die in der Datei „CrossSectionProfiles.inc“ vordefinierten (s.o.).

Probieren Sie aus, anstatt `HighwayCrossSection` in obiger Zuweisung die beiden anderen (`RuralMainRoadCrossSection` und `RuralRoadCrossSection`) für die Straße zu verwenden. Durchfahren Sie jeweils die Strecke.

Tipp:

Der vorgestellte `include` – Mechanismus funktioniert mit beliebigen Teilen der Skriptdatei. Damit lassen sich wiederverwendbare Teile eines Skripts in sog. `include` - Dateien „auslagern“ und in anderen Skriptdateien wieder einbinden. Wenn Sie z.B. schöne neue Straßenquerschnitte definieren, dann lohnt es sich, diese in eine solche `include` - Datei auszulagern (wie die von uns erstellte Datei „CrossSectionProfiles.inc“). Es empfiehlt sich, diesen `include` – Dateien die Endung „.inc“ zu geben. Damit ist erkennbar, dass es sich nur um einen Teil eines Skripts handelt, der für sich allein nicht in SILAB geladen werden kann.

4.4 Schritt 4: Landschaft

In den vorhergehenden Schritten dieses Tutorials haben wir die Straße fertiggestellt. Jetzt kümmern wir uns um die Ausgestaltung der umgebenden Landschaft.

In SILAB besteht die Landschaft aus zwei Teilen:

- Einem Höhenmodell, mit dem Hügel und Täler der Landschaft modelliert werden können.
- Objekten, die in der Landschaft platziert werden, z.B. Bäume, Häuser, Felder usw.

Selbst bei einer relativ kurzen Autobahnstrecke wie der unseren (Länge 5 km) wäre es sehr viel Arbeit, an jeder Stelle der Landschaft das Höhenmodell festzulegen und entlang der Straße links und rechts die Objekte einzeln zu platzieren. Hierfür bietet SILAB einen Mechanismus, der die Gestaltung von ansprechenden Landschaften viel einfacher macht: Landschaftsgeneratoren.

Laden Sie die Datei „tutorial_step_04_a.cfg“ in Ihren Editor. An der Markierung `### landscape` finden Sie folgendes:


```
### landscape
LandscapeTypeLeft Wooded
{
    Height = 3;
    Ripple = 0.7;
    Offset = 5;
    Gradient = -0.3;

    OverlayDensity = 0.8;
    TreeDensity = 90;
    MinObjdist = 15;
    MaxObjdist = 60;
};
LandscapeTypeRight Wooded
{
    Height = 1.5;
    Ripple = 1.1;
    Gradient = 0.2;

    OverlayDensity = 0.75;
    TreeDensity = 40;
    MinObjdist = 20;
    MaxObjdist = 150;
};
### /landscape
```

Bitte beachten Sie:

- Es wird die Landschaft für die linke Seite (`LandscapeTypeLeft`) und für die rechte Seite der Straße (`LandscapeTypeRight`) festgelegt.
- Auf beiden Seiten wird der Landschaftsgenerator `Wooded` benutzt. Er modelliert ein bewaldetes Gebiet entlang der Straße.
- In SILAB 4.0 gibt es neben `Wooded` 10 weitere Landschaftsgeneratoren für die unterschiedlichsten Landschaftsarten (z.B. bewirtschaftete oder besiedelte Gebiete, Weinberge, Alleen). Mit jedem Update von SILAB kommen i.d.R. neue dazu.
- Mit den Parametern `Height`, `Ripple`, `Offset` und `Gradient` wird das Höhenmodell der Landschaft beeinflusst.
- Die Parameter `TreeDensity`, `MinObjdist`, `MaxObjdist` beeinflussen die Verteilung von Objekten (im Fall des Landschaftsgenerators `Wooded` z.B. Bäume), der Parameter `OverlayDensity` die Bodenbeschaffenheit in der Landschaft.

Laden Sie die Datei in SILAB und fahren Sie. Aus den wenigen Zeilen im Skript wird eine Landschaft für den gesamten Verlauf der Straße erzeugt.



Wir beschäftigen uns zunächst mit dem **Höhenmodell** und betrachten beispielhaft die Parameter `Gradient`, `Height` und `Ripple`. Sie haben folgende Bedeutung:

<code>Gradient</code>	Mit diesem Parameter legen Sie fest, mit welcher Steigung die gesamte Landschaft mit zunehmendem Abstand von der Straße ansteigt oder abfällt. Der Wert misst den Höhenunterschied in Metern pro Meter Abstand von der Straße. In unserem Beispiel fällt die Landschaft auf der linken Seite der Straße pro Meter Abstand um 0.3 m ab (deswegen der negative Wert für den Parameter). Auf der rechten Seite steigt sie jeden Entfernungsmeter um 0.2 m an.
<code>Height</code>	Auf die generelle Steigung bzw. das generelle Gefälle, das der Parameter <code>Gradient</code> bestimmt, werden Täler und Hügel in Form einer zweidimensionalen Schwingung „überlagert“. Der Parameter <code>Height</code> legt die Amplitude dieser Schwingung fest, also die maximale Tiefe von Senken und die maximale Höhe von Hügeln (in Meter).
<code>Ripple</code>	Mit diesem Parameter beeinflussen Sie die Frequenz der Schwingung, also wie schnell sich Hügel und Senken in der Landschaft abwechseln.

Wir untersuchen den Einfluss der genannten Parameter auf die rechte Seite der Landschaft. Ändern Sie die Definition des `LandscapeTypeRight` wie folgt ab (rot markiert):

```
LandscapeTypeRight Wooded
{
    Height = 10.5;
    Ripple = 3.1;
    Gradient = 0;

    OverlayDensity = 0.75;
    TreeDensity = 40;
    MinObjdist = 20;
    MaxObjdist = 150;
};
```

Laden Sie die Datei in SILAB und fahren Sie die Straße entlang. Die Landschaft rechts steigt jetzt insgesamt nicht mehr an und ist sehr hügelig. Um das noch besser zu sehen, können Sie beim Fahren auch die Straße verlassen und durch die rechte Landschaft fahren. Probieren Sie einige andere Werte für die Parameter aus. Achtung: Zu extreme Einstellungen im Höhenmodell sehen sehr unrealistisch aus und können auch zu einer unruhigen Darstellung der Landschaftshöhen führen.

Die **Verteilung der Landschaftsobjekte** schauen wir uns auf der linken Seite an, hierzu insbesondere die Parameter `TreeDensity`, `MinObjdist` und `MaxObjdist`:

<code>TreeDensity</code>	Dieser Parameter legt fest, wie viele Bäume im Durchschnitt auf einer Fläche von 10.000 m ² (entspricht 1 Hektar) verteilt werden sollen.
<code>MinObjdist</code>	Die Landschaftsobjekte (in unserem Fall die Bäume) haben mindestens diesen Abstand zur Straße (in Meter).
<code>MaxObjdist</code>	Die Landschaftsobjekte haben maximal diesen Abstand zur Straße (in Meter).

Wir wollen unseren Wald auf der linken Seite etwas „lichter“ machen. Nehmen Sie hierzu folgende Änderungen vor (rot markiert):

```
LandscapeTypeLeft Wooded
{
    Height = 3;
    Ripple = 0.7;
    Offset = 5;
    Gradient = -0.3;

    OverlayDensity = 0.8;
    TreeDensity = 10;
    MinObjdist = 10;
    MaxObjdist = 500;
};
```

Wir haben folgendes gemacht:

- Durch die Reduktion der Baum-Dichte (`TreeDensity`) werden auf 1 Hektar nur noch ein neuntel der Bäume platziert als in der ursprünglichen Version.
- Dadurch wird der Wald durchsichtiger und der Fahrer kann u.U. die baumlose Landschaft dahinter erkennen. Deswegen haben wir auch den Abstand, in dem die Bäume maximal platziert werden, auf 500m erhöht (`MaxObjdist`).
- Wir haben den Waldrand etwas näher an die Straße verlegt (`MinObjdist`).

Experimentieren Sie mit diesen Parametern. Bitte beachten Sie, dass bei großen Werten von `TreeDensity` sehr viele Bäume dargestellt werden. Dies kann dazu führen, dass die Grafikleistung einbricht und die Simulation nicht mehr flüssig läuft.

Neben der allgemeinen, großflächigen Ausgestaltung der Landschaft mittels Landschaftsgeneratoren können Sie auch gezielt einzelne Objekte („Landmarks“) platzieren. Öffnen Sie die Datei „tutorial_step_04_b.cfg“ im Texteditor und suchen Sie die mit `### graphical objects` markierte Stelle:

```
### graphical objects
Objects =
{
    (house.Windmill, 2550, 100),
    (house.Windmill, 2650, 110),
    (house.Windmill, 2750, 120)
};
### /graphical objects
```

Mit diesem Abschnitt werden drei Windkraftanlagen eingefügt. Allgemein gesagt gilt folgendes:

- Alle einzeln platzierten graphischen Objekte werden als Elemente der Menge `Objects = { ... };` aufgeführt. Jedes Element entspricht einem Objekt.
- Ein Element hat – im einfachsten Fall – die Form

`(<Objektname>, <Streckenmeter>, <Abstand>)`

- Der `<Objektname>` bezieht sich auf Dateien, in denen SILAB die Informationen zu einem grafischen Objekt finden kann. Im Lieferumfang von SILAB ist eine große Bibliothek solcher grafischen Objekte enthalten. Sie befinden sich im Unterordner „lib\graphics\default\models“ („Modellordner“) Ihres SILAB Installationsverzeichnis. Der `<Objektname>` ist der Pfad zu einer solchen Datei relativ zum Modellordner. Das Zeichen „\“ wird durch einen Punkt ersetzt, die Dateinamenserweiterung „*.sig“ oder „*.sge“ wird weggelassen. Unsere Windkraftanlagen sind also in der Datei „lib\graphics\default\models\house\Windmill.sig“ definiert.
- `<Streckenmeter>` legt die Position entlang der Straßen fest, gemessen in Meter vom Beginn des ebenen Straßenverlaufs.
- `<Abstand>` legt fest, wie weit die Objekte von der exakten Straßenmitte entfernt sind (in Meter). Positive Werte liegen rechts von der Straßenmitte, negative links davon.

Laden Sie die Datei in SILAB. Nach ca. 1,5 km Fahrt können Sie die Windkraftanlagen sehen.



4.5 Schritt 5: Ausbau des Straßennetzes

In diesem Schritt bereiten wir das Straßennetz für die simulierten Verkehrsteilnehmer vor. Die Straße, auf der sich das eigentliche Folgefahrt-Szenario abspielen soll, ist bereits fertig. Zu Beginn der Simulation müssen wir dem Fahrer aber Zeit geben, sich an das Fahrzeug, dem er folgen soll, anzunähern und eine „stabile“ Folgefahrt herzustellen. Hierfür benötigen wir noch ein kurzes Autobahn-Stück, in dem das Szenario gestartet wird. Dieses Stück definieren wir in diesem Schritt des Tutorials.

Öffnen Sie die Datei „tutorial_step_05.cfg“ und suchen Sie den mit `### courses` markierten Abschnitt. Wenn man einige Details außer Acht lässt, dann sieht dieser Abschnitt jetzt so aus:

```

### courses
define Course Start
{
    NodeID = 0;

    ### cross section assignment
    CrossSection = HighwayCrossSection;
    ### /cross section assignment

    ### planar road geometry
    Straight(1000);
    ### /planar road geometry

    ### landscape
    (der Übersichtlichkeit halber weggelassen)
    ### /landscape
};

define Course HighwayRoad
{
    (der Übersichtlichkeit halber weggelassen)
};
### /courses

```

Bitte beachten Sie:

- Die Definition eines Straßenstücks erfolgt in SILAB mit

```

define Course <Name>
{
    ...
};

```

- Unsere Datei enthält zwei solche Straßenstücke mit <Name>
 - Start: Unser neues Stück für den Start des Szenarios.
 - HighwayRoad: Das Straßenstück, das wir in den bisherigen Schritten fertiggestellt haben.
- Innerhalb eines jeden solchen Straßenstücks werden die Ihnen bereits bekannten Informationen festgelegt: Verlauf der Straße in der Ebene (s. 4.1), Höhenprofil (s. 4.2), Querschnittsprofil (s. 4.3) und Landschaft (s. 4.4). Für den Straßenabschnitt `HighwayRoad` wurden diese Informationen unverändert aus den vorhergehenden Schritten des Tutorials übernommen.

Versuchen Sie die Informationen im neuen Straßenstück `Start` nachzuvollziehen, insbesondere folgendes:

- Es wird das gleiche Querschnittsprofil namens `HighwayCrossSection` benutzt wie im Straßenstück `HighwayRoad`. Die beiden Abschnitte sollen ja schließlich „nahtlos“ aneinander passen.
- Der Verlauf der Straße in der Ebene ist absichtlich sehr einfach und für den Fahrer übersichtlich gestaltet. Er besteht lediglich aus einer Geraden von 1 km Länge.
- Ebenfalls aus Gründen der Übersichtlichkeit verzichten wir vollständig auf ein Höhenprofil. Die Straße ist also vollkommen eben.

- Um ein bisschen Abwechslung zu erzeugen, wurde als Landschaftsgenerator für die linke und rechte Seite der Straße `Farmed` gewählt, was einem bewirtschafteten Gebiet mit Feldern und einzelnen Gehöften entspricht.

Die beiden Straßenstücke `start` und `HighwayRoad` müssen wir jetzt noch verbinden. Das geschieht an der Stelle, die mit `### road network` markiert ist:

```
### road network
Start start;
HighwayRoad road;
Connections =
{
    start.End <-> road.Begin,

    start.Begin <-> Port1,
    road.End <-> Port2
};
### /road network
```

In den rot markierten Zeilen werden die eigentlichen Straßenstücke erzeugt. Genau genommen haben wir nämlich mit den Abschnitten `define Course Start { ... };` und `define Course HighwayRoad { ... };` nur **Typen** von Straßenstücken erstellt. Die tatsächlich befahrbaren Straßenstücke `start` und `road` (s. rot markierten Text) sind sog. **Instanzen** der jeweiligen Typen (mehr dazu weiter unten in diesem Abschnitt).

In `Connections = { ... };` werden die Instanzen dann zu einem Straßennetz verknüpft. In unserem Fall wird das Ende von `start` (`start.End`) mit dem Anfang von `road` (`road.Begin`) verbunden (blau markiert). Die beiden schwarzen Zeilen darunter interessieren uns im Moment nicht.

Laden Sie die Datei in SILAB und fahren Sie. Sie werden auf dem neuen Straßenstück `start` aufgesetzt und gelangen nach 1 km auf unser bekanntes Stück `road`.



Vielleicht kennen Sie das oben erwähnte Konzept von Typen und Instanzen aus einer Programmiersprache. In Java gibt es beispielsweise den Datentyp `int` für eine

Zahl. Um eine Variable `x` dieses Typs in Ihrem Programm verwenden zu können, erzeugen Sie eine Instanz des Datentyps `int`. Wenn Sie keine Programmiererfahrung haben, ist das Konzept wahrscheinlich etwas verwirrend. Das folgende Beispiel hilft Ihnen hoffentlich, den Vorteil des Konzepts zu erkennen.

Angenommen, uns ist das Straßenstück `road`, in dem sich ja unser Folgefahrt-Szenario abspielen soll, zu kurz. Die einfachste Möglichkeit, die Strecke zu verlängern, ist, sie einfach mehrmals hintereinander zu hängen. Um z.B. drei Wiederholungen zu erzeugen, müssen Sie einfach drei Instanzen von `HighwayRoad` anlegen und diese zu einem Straßennetz verknüpfen. Ändern Sie hierzu den Abschnitt `### road network` wie folgt ab:

```
### road network
Start start;
HighwayRoad road1;
HighwayRoad road2;
HighwayRoad road3;
Connections =
{
    start.End <-> road1.Begin,
    road1.End <-> road2.Begin,
    road2.End <-> road3.Begin,

    start.Begin <-> Port1,
    road3.End <-> Port2
};
### /road network
```

Es gibt jetzt drei Instanzen (`road1`, `road2`, `road3`) unseres Typs `HighwayRoad`, die im Abschnitt `Connections = { ... };` aneinander gehängt werden. Es entsteht eine 15 km lange Autobahnstrecke (`start` nicht mitgerechnet).

Noch eine Bemerkung zum Höhenprofil von Straßen: In Abschnitt 4.2 haben Sie gelesen, dass das erste und das letzte Segment eines Höhenprofils immer die Steigung 0% haben muss. Den Grund dafür erkennen Sie jetzt: Würde unsere Straße `HighwayRoad` z.B. mit einer Steigung enden und würde man wie oben mehrere Instanzen davon „glatt“ aneinanderhängen, dann würden sich diese Steigungen aufsummieren. Im Extremfall ergäben sich dadurch sogar Loopings.

4.6 Schritt 6: Vorausfahrendes Fahrzeug

Nach diesen Vorbereitungen können wir jetzt das vorausfahrende Fahrzeug, dem der Fahrer im gesamten Szenario folgen soll, einfügen. Es soll sich wie folgt verhalten:

- Beim Start der Simulation, bei dem der Fahrer ja auf dem Straßenstück `start` (s. vorhergehender Abschnitt) aufgesetzt wird, soll das Fahrzeug in einiger Entfernung voraus warten.
- Sobald der Fahrer losfährt, beschleunigt auch das vorausfahrende Fahrzeug. Dies soll so langsam geschehen, dass der Fahrer auf jeden Fall Anschluss halten kann.
- Wenn das vorausfahrende Fahrzeug die Zielgeschwindigkeit von 80 km/h erreicht hat, soll es diese Geschwindigkeit beibehalten.

Öffnen Sie die Datei „tutorial_step_06.cfg“. Um simulierte Verkehrsteilnehmer zu definieren, müssen wir zunächst einen Abschnitt einfügen, der allgemeine Einstellungen dafür festlegt. Suchen Sie in der Datei die mit `### component configuration` markierte Stelle.

```
### component configuration
includeif mov\mop_700.inc
include sge\sge_700.cfg
include scnx\scnxsge_700.cfg
includeif sndx\sndx.cfg

SILAB TRF
{
    include SILAB_TRFX_700.cfg
};
### /component configuration
```

Der Abschnitt mit den allgemeinen Einstellungen für die Verkehrssimulation ist rot markiert. In den allermeisten Fällen reicht es, wenn Sie einfach die im Lieferumfang von SILAB enthaltene Datei „SILAB_TRFX_700.cfg“ einbinden (mit `include`). In ihr sind die wichtigsten Einstellungen enthalten, u.a. eine Liste der gängigsten Fahrzeugtypen.

Suchen Sie die mit `### traffic` markierte Stelle.

```
### road network
(der Übersichtlichkeit halber weggelassen)
### /road network

### traffic
Traffic MyTraffic
{
    ### leading vehicle
    (der Übersichtlichkeit halber weggelassen)
    ### /leading vehicle
};
### /traffic
```

Bitte beachten Sie:

- Die Definition des Verkehrs erfolgt innerhalb des Abschnitts

```
Traffic <beliebiger Name>
{
    ...
};
```

- Da sich alle simulierten Verkehrsteilnehmer im vorher definierten Straßennetz bewegen sollen, befindet sich dieser Abschnitt im Skript auf der gleichen Ebene wie der Abschnitt `### road network` (und nicht etwa innerhalb eines `define Course <Name> { ... };` Abschnitts).

Innerhalb des Traffic Abschnitts finden Sie an der mit `### leading vehicle` markierten Stelle die Definition des vorausfahrenden Fahrzeugs:

```
### leading vehicle
VehicleX LeadingVehicle
{
    Vehicle = OpelZafira_white;
    Position = (start, 50.0, 5);
    BehaviourScheme = Standard;
    StartSpeed = 0.0;
    TargetSpeed = 0.0;
    Flowpoints =
    {
        (ModuleTime, 0, Activate),
        (start, 1, 6, SimCar, TargetSpeedAndAcceleration, 22.22, 1.0)
    };
};
### /leading vehicle
```

Diesen Abschnitt schauen wir jetzt genauer an:

- Er hat die Form

```
VehicleX <Name>
{
    ...
};
```

Der `<Name>` ist dabei beliebig.

- Mit der Zuweisung `Vehicle = <Fahrzeugtyp>`; legen Sie fest, welcher Fahrzeugtyp angezeigt werden soll, in unserem Fall ein weißer Opel Zafira.
- Es folgt die anfängliche Position des Fahrzeugs, festgelegt durch

```
Position = (<Straßenstück>, <Streckenmeter>, <Spurnummer>);
```

Unser vorausfahrendes Fahrzeug wird also auf dem Straßenstück `start` in einer Entfernung von 50 m entlang der Straße auf der Spur 5 erzeugt. Die Spurnummern entsprechen denen aus dem Querschnittsprofil (s. Abschnitt 4.3).

- Jeder simulierte Verkehrsteilnehmer verhält sich gemäß bestimmter Verhaltensmodelle. In SILAB gibt es sehr viele solche Verhaltensmodelle für die unterschiedlichsten Aufgabenstellungen. In unserem Fall reicht uns das Standard-Verhalten aus. Es wird mit

```
BehaviourScheme = Standard;
```

ausgewählt.

- Jedes Verhaltensmodell hat Parameter, mit denen es beeinflusst werden kann. Beispielsweise hat das Verhaltensmodell `Standard` Parameter, mit denen die Geschwindigkeit des Verkehrsteilnehmers festgelegt wird. Unser Vorausfahrender soll zunächst warten, deswegen wird sowohl der Parameter

`StartSpeed` für die anfängliche Geschwindigkeit als auch der Parameter `TargetSpeed` für die „Wunschgeschwindigkeit“ auf 0 m/s gesetzt.

- Es folgt ein Abschnitt `Flowpoints = { ... }`; der sog. Flowpoints enthält. Mit Flowpoints kann das Verhalten eines Verkehrsteilnehmers im Verlauf eines Szenarios beeinflusst werden. In unserem Fall gibt es zwei davon.
- Der erste Flowpoint

```
(ModuleTime, 0, Activate)
```

ist zeitbasiert. Er wird ausgelöst, wenn eine Zeit von 0 Sekunden seit der Aktivierung des Szenarios vergangen ist, also unmittelbar beim Start der Simulation. Es folgt die Definition, was dann geschehen soll. In unserem Fall sorgt das Kommando `Activate` dafür, dass das vorausfahrende Fahrzeug erzeugt wird.

- Der zweite Flowpoint

```
(start, 1, 6, SimCar, TargetSpeedAndAcceleration, 27.78, 1.0)
```

ist streckenbasiert. Sie können sich einen streckenbasierten Flowpoint als unsichtbaren Punkt auf einer bestimmten Spur vorstellen. Wenn wahlweise der Fahrer oder ein simulierter Verkehrsteilnehmer diesen Punkt „überfährt“, wird er ausgelöst. Er hat die Form

```
(<Streckenstück>, <Streckenmeter>, <Spurnummer>, <Kontext>, <Kommando>, <Parameter>)
```

In unserem Fall befindet er sich im Streckenabschnitt `start` auf Streckenmeter 1 (also beim Start der Simulation unmittelbar vor dem Fahrer) und auf Spur 6 (das ist der Standstreifen, auf dem sich der Fahrer beim Start der Simulation befindet). Mit dem `<Kontext>` legen Sie fest, wer den Flowpoint auslöst. Der Kontext `SimCar` bedeutet, dass der Flowpoint vom Fahrer ausgelöst wird. Das Kommando `TargetSpeedAndAcceleration` hat zwei Parameter: Die Wunschgeschwindigkeit (bei uns: 27.78 m/s = 100 km/h) und die Beschleunigung zum Erreichen der Wunschgeschwindigkeit (bei uns: gemächliche 1 m/s²).

Laden Sie die Datei in SILAB und vollziehen Sie das Verhalten des Vorderfahrzeugs nach.



Experimentieren Sie, in dem Sie

- statt `OpelZafira_white` andere Fahrzeugtypen wählen, z.B.

```
VWPassat
Alfa147
FusoCanter
MAN_TGL
BMW1200
Mountainbike
```

- Die anfängliche Position mit `Position = (start, 10.0, 5);` näher zum Fahrer verlegen.
- Mit `(ModuleTime, 5, Activate)` das Vorderfahrzeug erst 5 Sekunden nach dem Start erscheinen lassen.
- Das Vorderfahrzeug mit

```
(start, 1, 6, SimCar, TargetSpeedAndAcceleration, 27.78, 7.0)
```

wie einen Sportwagen in ca. 4 Sekunden von 0 auf 100 km/h beschleunigen lassen.

4.7 Schritt 7: Mitverkehr

Wichtig für unser Szenario ist, dass der Fahrer das vorausfahrende Fahrzeug nicht überholt. Das verhindern wir (neben einer entsprechenden Instruktion des Fahrers vor der Fahrt), in dem wir auf der Spur links neben dem Fahrer Verkehr erzeugen, der ihn mit relativ hoher Geschwindigkeit überholt.

Es wäre sehr mühsam, die vielen Fahrzeuge dieses überholenden Verkehrs über die gesamte Länge unserer Autobahn einzeln im Skript zu definieren. Deswegen bietet SILAB für diese Art von stochastischen Verkehrsflüssen einen praktischen Mechanismus: Auf der linken „Mit“-Spur wird eine sog. Fahrzeugquelle eingefügt, die mit einem bestimmten Abstand hinter dem Fahrer her fährt. Diese Fahrzeugquelle erzeugt in zufälligen Zeitabständen Fahrzeuge, die den Fahrer überholen. Mit dem Fahrer fährt eine sogenannte Fahrzeugsenke mit. Diese Senke deaktiviert die von der Quelle erzeugten Fahrzeuge, wenn sie weiter als ein bestimmter Abstand vom

EGO-Fahrzeug entfernt sind. Auf diese Weise entsteht auf der Spur links vom Fahrer im gesamten Szenario ein konstanter Strom von Fahrzeugen.

Öffnen Sie die Datei „tutorial_step_07.cfg“ im Texteditor und suchen Sie den Abschnitt `### overtaking traffic`.

```
### overtaking traffic
TrafficFlow OvertakingTraffic
{
    Source OTSource
    {
        Position =(SimCar, -500, 4);
        Vehicles =
        {
            (15, Car, (HazardAvoidance, 1.0), (FreeDriving, 35, 35))
        };
        Parameters = (Gauss, 3.0, 1.4, 451,512);
    };

    Drain OTDrain
    {
        Position = (SimCar, 1000, 4);
    };

    Flowpoints =
    {
        (start, 500, 5, SimCar, ActivateSource, (OTSource)),
        (start, 500, 5, SimCar, ActivateDrain, (OTDrain))
    };
};
### /overtaking traffic
```

Bitte beachten Sie:

- Ein Verkehrsfluss wird in einem Abschnitt der Form

```
TrafficFlow <Name>
{
    ...
};
```

definiert. Dieser Abschnitt befindet sich im Skript auf der gleichen Ebene wie der Abschnitt für unser einzelnes vorausfahrendes Fahrzeug. Ein Verkehrsfluss ist also genau wie Einzelfahrzeuge ein sog. Verkehrsobjekt.

- Innerhalb dieses Abschnitts wird in einem Unterabschnitt `Source <Name> { ... };` zunächst die Fahrzeugquelle definiert.
 - Die Quelle wird mit `Position = ...;` positioniert. Unsere Quelle wird mit einem Abstand von 500 m auf Spur 4 hinter dem Fahrer platziert (negative Abstände liegen in Fahrtrichtung hinten, positive vorne).
 - In `Vehicles = { ... };` wird festgelegt, dass die Quelle 15 Fahrzeuge zur Verfügung hat, die sie aus der Gruppe namens `Car` zufällig zieht. Die Gruppe `Car` enthält verschiedene PKWs. Jedes von der Quelle erzeugte Fahrzeug verhält sich nach den gleichen Verhaltensmodellen, die im Anschluss definiert werden (`HazardAvoidance` zur Vermeidung

von Auffahrunfällen und `FreeDriving` zum Fahren mit einer bestimmten Geschwindigkeit). Die Fahrzeuge halten einen Mindestabstand von 1 Sekunde zueinander (Parameter von `HazardAvoidance`) und fahren mit einer Geschwindigkeit von 126 km/h = 35 m/s (Parameter von `FreeDriving`; 1. Parameter = Startgeschwindigkeit, 2. Parameter = Zielgeschwindigkeit).

- Die Zufallsfolge der Abstände kann mit `Parameters = ( ... );` beeinflusst werden. Die Fahrzeuge werden mit normalverteilten (`Gauss`) zeitlichen Abständen erzeugt. Der Mittelwert der Abstände beträgt 3 Sekunden, die Standardabweichung 1.4 s.
- Es folgt die Definition der Senke im Unterabschnitt `Drain <Name> { ... };` Die einzige Angabe hier ist der Abstand vom Fahrer, ab dem die Fahrzeuge der Quelle wieder deaktiviert werden. Dieser Abstand wird auf 1000 m gesetzt. Die Spurnummer 4 wird ignoriert, da die mitfahrende Senke ja einen Radius um den Fahrer darstellt.
- Wie schon beim vorausfahrenden Fahrzeug folgt ein Abschnitt mit `Flowpoints`. Es gibt zwei streckenbasierte, die auf derselben Position liegen (Streckenstück `start`, Streckenmeter 500, Spur 5) und vom Fahrer ausgelöst werden (Kontext `SimCar`). Der erste Flowpoint aktiviert die Quelle, der zweite aktiviert die Senke.

Laden Sie die Datei in SILAB und beobachten Sie, wie sich der Verkehrsfluss auf der Spur links von Ihnen nach einiger Zeit aufbaut. Er ist ungefähr dann in vollem Gange, wenn Sie vom Streckenstück `start` auf das Streckenstück `road` gelangen (erkennbar an der Waldlandschaft).

Experimentieren Sie:

- Wählen Sie im Abschnitt `Source OTSource = { ... };` als zusätzliche Fahrzeuggruppe `TruckMotorway`:

```
Vehicles =
{
    (15, Car, (HazardAvoidance, 2), (FreeDriving, 15, 35)),
    (5, TruckMotorway, (HazardAvoidance, 2), (FreeDriving, 15, 35))
};
```

Die Quelle zieht jetzt die Fahrzeuge, die sie erzeugt, aus den Gruppen `Car` und `TruckMotorway`. Beobachten Sie, wie Sie jetzt auch von LKWs (mit halsbrecherischen Geschwindigkeiten) überholt werden.

- Erhöhen Sie die Geschwindigkeit des Verkehrsflusses auf 180 km/h (50 m/s):

```
Vehicles =
{
    (15, Car, (HazardAvoidance, 1.0), (FreeDriving, 50, 50))
};
```

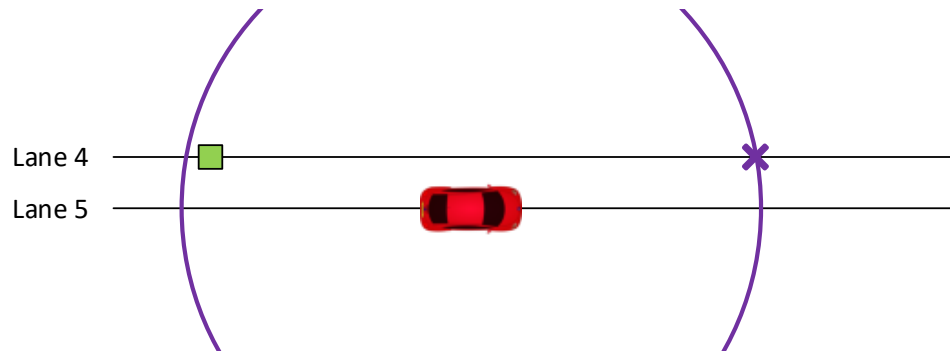
- Verringern Sie den Abstand der Quelle auf 20 m

```
Position = (SimCar, -20, 4);
```

und den Radius der Senke auf 30 m

```
Position = (SimCar, 30, 4);
```

Beobachten Sie, wie die Fahrzeuge deaktiviert werden, die sich in 30 m Entfernung vor dem Fahrer befinden. Die Situation ist auf dem folgenden Bild dargestellt:



Die Quelle (grün) fährt im Abstand von 20 m hinter dem Fahrer auf Spur 4. Die Senke (lila Kreis) deaktiviert die Fahrzeuge, sobald sie weiter als 30 m vom Fahrer entfernt sind (lila Kreuz). Anhand des Bildes sehen Sie auch, dass der Radius der Senke immer größer sein muss als der Abstand der Quelle.

4.8 Schritt 8: Gegenverkehr

Auf den Spuren 1 und 2 unserer Autobahn fahren noch keine Fahrzeuge. Das ist natürlich unrealistisch. Wir möchten leichtes Verkehrsaufkommen in der Gegenrichtung. Auch hierfür setzen wir Verkehrsflüsse ein, für jede Spur in Gegenrichtung einen.

Öffnen Sie die Datei „tutorial_step_08.cfg“ im Texteditor und suchen Sie den Abschnitt, der mit `### oncoming traffic` markiert ist.

```

### oncoming traffic
TrafficFlow OncomingTraffic1
{
    Source OCTSource1
    {
        (der Übersichtlichkeit halber weggelassen)
    };

    Drain OCTDrain1
    {
        (der Übersichtlichkeit halber weggelassen)
    };

    Flowpoints =
    {
        (der Übersichtlichkeit halber weggelassen)
    };
};

TrafficFlow OncomingTraffic2
{
    Source OCTSource2
    {
        (der Übersichtlichkeit halber weggelassen)
    };

    Drain OCTDrain2
    {
        (der Übersichtlichkeit halber weggelassen)
    };

    Flowpoints =
    {
        (der Übersichtlichkeit halber weggelassen)
    };
};
### /oncoming traffic

```

Analysieren Sie den Abschnitt und beachten Sie dabei bitte insbesondere:

- Der Verkehrsfluss `TrafficFlow OncomingTraffic1` ist für den Verkehr auf Spur 1, d.h. der vom Fahrer aus gesehen ganz linken Fahrspur (nicht Standstreifen) zuständig. Der Verkehrsfluss `TrafficFlow OncomingTraffic2` ist für die Spur 2 zuständig. Das erkennen Sie an der Positionierung (`Position = ...;`) der jeweiligen Quellen `OCTSource1` und `OCTSource2`.
- Für die aus Fahrersicht ganz linke Spur erzeugt die Quelle `OCTSource1` sowohl PKWs als auch LKWs, die mit ca. 80 km/h (22 m/s) fahren.
- Für die Spur daneben werden nur PKWs mit einer Geschwindigkeit von 140 km/h erzeugt.
- Der Verkehr von `OncomingTraffic1` ist etwas dichter als der von `OncomingTraffic2` (s. `Parameters = ...;` in den Quellen).

4.9 Schritt 9: Datenaufzeichnung (DPU-Erweiterungen)

Für jeden Fahrer, der unser Szenario später einmal fahren wird, möchten wir Daten zu dessen Fahrverhalten aufzeichnen. Diese Daten benötigen wir für eine nachträgliche Analyse.

Öffnen Sie die Datei „tutorial_step_09.cfg“ und suchen Sie den Abschnitt `### extended DPU pool`.

```
# use default DPU pool
include system\DPUBase_SGE.inc

### extended DPU pool
Pool MyDPUPool : Full
{
    Executable = true;

    # information on traffic participants
    include system\DPUTRFDataFull.inc

    ### data recording
    (der Übersichtlichkeit halber weggelassen)
    ### /data recording
};
### /extended DPU pool
```

Hier geschieht folgendes:

- Eine Fahrsimulation mit SILAB besteht aus vielen einzelnen Softwarekomponenten, von denen jede eine spezielle Aufgabe hat. Es gibt z.B. Softwarekomponenten für die physikalische Simulation des Fahrzeugs, für die Geräuschsimulation, für die Simulation der anderen Verkehrsteilnehmer und für die Bildgenerierung.
- Eine solche Softwarekomponente heißt in SILAB „DPU“ (Data Processing Unit). Ähnlich wie elektronische Bauteile besitzen DPUs bestimmte Eingänge. In jedem Simulationsschritt erfüllen sie – basierend auf den Werten an ihren Eingängen – ihre jeweiligen Aufgaben. Beispielsweise hat die DPU für die Geräuschsimulation einen Eingang für die Motordrehzahl und die Gaspedalstellung. Abhängig davon erzeugt sie das Motorgeräusch.
- Die DPUs tauschen untereinander Daten aus. Beispielsweise bezieht die DPU für die Geräuschsimulation ihre Eingangswerte von der DPU, die das physikalische Modell des Fahrzeugs berechnet.
- Die Auflistung der benötigten DPUs und die Festlegung des Datenaustausches zwischen ihnen heißt „DPU Pool“ oder nur „Pool“.
- In unserem Beispiel sehen Sie die Zeilen

```
# use default DPU pool
include system\DPUBase_SGE.inc
```

Hier wird (mit dem `include` Mechanismus) eine Datei aus dem Lieferumfang von SILAB eingebunden, in der der komplette grundlegende DPU Pool Ihres Simulators enthalten ist.

- Wenn Sie für Ihr Szenario zusätzliche DPUs benötigen (z.B. Fahrerassistenzsysteme, Nebenaufgaben für den Fahrer oder – wie in unserem Beispiel – eine Datenaufzeichnung), dann können Sie den grundlegende DPU Pool entsprechend erweitern. Im Beispiel sieht das so aus:

```
### extended DPU pool
Pool MyDPUPool : Full
{
    (der Übersichtlichkeit halber weggelassen)
};
### /extended DPU pool
```

Es wird ein neuer DPU Pool namens `MyDPUPool` erzeugt, der den grundlegenden DPU Pool namens `Full` erweitert.

- Die Skript-Abschnitte einiger besonders häufig benötigter Erweiterungen sind ebenfalls im Lieferumfang von SILAB enthalten. Dazu gehört z.B. ein Skript-Abschnitt mit einer DPU `TRFData`, die detaillierte Information über die Verkehrsteilnehmer in der Umgebung des Fahrers liefert. Diese Erweiterung wird in unserem Pool `MyDPUPool` so eingebunden:

```
# information on traffic participants
include system\DPUTRFDataFull.inc
```

- Schließlich wird die DPU konfiguriert, die für die Datenaufzeichnung verantwortlich ist. Das schauen wir uns jetzt genauer an.

Suchen Sie den Abschnitt `### data recording` in unserem DPU Pool `MyDPUPool`.

```
### data recording
DPUDataFile DataLogger
{
    Computer = {DATA};
    Index = 1002;
    Separator = ",";
    HeaderType = 1;
    Record =
    {
        Mockup.AcceleratorPedal ("AcceleratorPedal"),
        Mockup.BrakePedal ("BrakePedal"),
        VDyn.v_kmh ("v_kmh"),
        VDyn.ax ("ax"),
        VDyn.ay ("ay"),
        SCN.NodeTemplateID ("NodeTemplateID"),
        SCN.s ("s"),
        SCN.LaneWidth ("LaneWidth"),
        SCN.LateralDistance ("LateralDistance"),
        TRFData.StLongSameAheadNext ("d"),
        TRFData.vSameAheadNext ("v")
    };
};
### /data recording
```

Bitte beachten Sie:

- Wir verwenden eine Instanz des DPU-Typs `DPUDataFile`. Als Instanzname wählen wir `DataLogger` (blau markiert). Das Konzept von Typ und Instanz kennen Sie ja bereits aus Abschnitt 4.5.
- Mit `Computer = {DATA};` wird festgelegt, dass die DPU vom Computer namens `DATA` berechnet werden soll. Je nach Aufbau Ihres Simulators haben wir bei der Installation von SILAB festgelegt, welcher „physikalische“ Rechner das ist.
- Neben unserer Datenaufzeichnung hat der Rechner `DATA` noch andere DPUs auszuführen. Die Position der DPU `DataLogger` in der Berechnungsreihenfolge wird über `Index = 1002;` festgelegt. Der hohe Wert bedeutet, dass die DPU nach allen anderen DPUs an der Reihe ist.
- Der Parameter `Separator = „“;` legt fest, dass als Trennzeichen für die einzelnen Werte in der Aufzeichnungsdatei ein Komma verwendet werden soll.
- `HeaderType = 1;` legt fest, dass über jeder Spalte der Aufzeichnungsdatei der Name des entsprechenden Signals stehen soll.
- Im Abschnitt `Record = { ... };` wird schließlich festgelegt, welche Daten aufgezeichnet werden sollen. Jede Zeile in diesem Abschnitt entspricht einem bestimmten Messwert und hat die Form

```
<DPU-Name>.<Ausgang> („<Name in Aufzeichnungsdatei>“)
```

- Beispielsweise finden Sie die Zeile

```
Mockup.AcceleratorPedal („AcceleratorPedal“),
```

Sie legt fest, dass die Gaspedalstellung aufgezeichnet werden soll. Dieser Wert wird von der DPU namens `Mockup` zur Verfügung gestellt, und zwar in einem ihrer Ausgänge namens `AcceleratorPedal`. Die DPU `Mockup` ist Bestandteil des grundlegenden DPU Pools, den wir ja gerade erweitern. Wir wollen, dass der Wert in der Aufzeichnungsdatei auch `AcceleratorPedal` heißt.

- Ein weiteres Beispiel:

```
VDyn.ax („ax“),
```

Es wird die Längsbeschleunigung aufgezeichnet. Sie wird von der DPU `VDyn` als Ausgang zur Verfügung gestellt. Die DPU `VDyn` berechnet das physikalische Fahrzeugmodell. Die Spalte in der Aufzeichnungsdatei soll `ax` heißen.

- Die restlichen aufgezeichneten Werte sind:

```
Mockup.BrakePedal ("BrakePedal")
    Bremspedalstellung von 0 bis 1.
VDyn.v_kmh ("v_kmh")
    aktuelle Geschwindigkeit des Fahrers in km/h
VDyn.ay ("ay")
    Querschleunigung des Fahrers in m/s²
SCN.NodeTemplateID ("NodeTemplateID")
    Kennung des Straßenabschnitts (mehr dazu im nächsten Kapitel)
SCN.s ("s")
    Streckenmeter auf dem aktuellen Straßenabschnitt in m
```

`SCN.LaneWidth ("LaneWidth")`
 Breite der aktuellen Fahrspur in m
`SCN.LateralDistance ("LateralDistance")`
 Abstand von der exakten Mitte der aktuellen Fahrspur („Ideal-Linie“) in
 Meter. Negative Werte liegen links von der Ideal-Linie, positive rechts
 davon.
`TRFData.StLongSameAheadNext ("d")`
 Abstand zum vorausfahrenden Fahrzeug (Stoßstange – Stoßstange) in
 m
`TRFData.vSameAheadNext ("v")`
 Geschwindigkeit des vorausfahrenden Fahrzeugs in m/s

Laden Sie die Datei in SILAB, wählen Sie den DPU-Pool „MyDPUPool“ und fahren Sie ein Stück. Beenden Sie SILAB und werfen Sie einen Blick in den Unterordner „data“ Ihres SILAB Installationsverzeichnis. Sie sehen folgende Dateien:

DataFile1.asc	Das ist die eigentliche Aufzeichnungsdatei, die für die Fahrt erzeugt wurde. Mit ihr befassen wir uns im nächsten Kapitel.
DataFile1.cfg	Zu jeder Aufzeichnungsdatei speichert SILAB nochmal eine Kopie der zugehörigen Skriptdatei. Das ist sehr praktisch, wenn Sie zu einem späteren Zeitpunkt z.B. wissen möchten, welche Strecke der Aufzeichnung zugrunde liegt.
DataFile.XXX.asc.num	Eine interne Datei von SILAB, die das Überschreiben gleich oder nicht benannter Aufzeichnungsdateien verhindert (s.u.).

Fahren Sie erneut ein Stück auf unserer Autobahn. Wenn Sie jetzt in das Verzeichnis „data“ schauen, sehen Sie zwei weitere Dateien, „DataFile2.asc“ und „DataFile2.cfg“. SILAB überschreibt keine bereits existierenden Aufzeichnungsdateien. Stattdessen wird am Ende des Dateinamens ein Zähler erhöht. Selbstverständlich gibt es Möglichkeiten, den Dateinamen vor jedem Start der Simulation einzugeben. Im Rahmen dieses Tutorials wollen wir darauf aber nicht eingehen.

4.10 Schritt 10: Test und Auswertung

In diesem Abschnitt befassen wir uns mit dem Inhalt der Aufzeichnungsdatei. Laden Sie bitte hierzu die Datei „tutorial_step_09.cfg“ in SILAB. Bitte durchfahren Sie diesmal die gesamte Strecke und versuchen Sie, dem Vorderfahrzeug möglichst realistisch zu folgen.

Nach der Fahrt hat SILAB im Unterordner „data“ Ihres SILAB Installationsverzeichnis eine weitere Aufzeichnungsdatei erzeugt, z.B. „DataFile3.asc“. Ihre Größe sollte einige MB betragen. Öffnen Sie diese Datei in Excel oder in Ihrem bevorzugten Statistik-Programm. Wenn Sie Excel verwenden, erscheint nach der Auswahl der Datei der „Textkonvertierungs-Assistent“. Nehmen Sie folgende Einstellungen in den 3 Schritten dieses Assistenten vor:

Schritt 1: „Getrennt“ muss ausgewählt sein.

Schritt 2: Wählen Sie als Trennzeichen „Komma“.

Schritt 3: Klicken Sie auf „Weitere ...“ und wählen als Dezimaltrennzeichen den Punkt und als 1000er-Trennzeichen das Hochkomma.

Nach dem Import erscheint folgende Tabelle:

A	B	C	D	E	F	G	H	I	J
MeasurementTime	Measurement time error	AcceleratorPedal	BrakePedal	v_kmh	ax	ay	NodeTemplateID	s	LaneWidth
0	0	0	0	0	0	0	0	0	3
10	0	0	0	0	-0,000027	0	0	0	2,5
20	0	0	0	0	-0,000103	0	0	0	2,5
30	0	0	0	0	-0,000185	0	0	0	2,5
40	0	0	0	0	-0,000217	0	0	0	2,5
50,01	0,01	0	0	0	-0,000192	0	0	0	2,5
60	0	0	0	0	-0,000126	0	0	0	2,5
70,01	0,01	0	0	0	-0,000039	0	0	0	2,5
80,01	0,01	0	0	0	0,000005	0	0	0	2,5
90	0	0	0	0	0,000128	0	0	0	2,5
100	0	0	0	0	0,000187	0	0	0	2,5
110,05	0,05	0	0	0	0,000224	0	0	0	2,5
120	0	0	0	0	0,000242	0	0	0	2,5
130,01	0,01	0	0	0	0,000242	0	0	0	2,5
140,01	0,01	0	0	0	0,000226	0	0	0	2,5
150,01	0,01	0	0	0	0,000199	0	0	0	2,5
160,01	0,01	0	0	0	0,000163	0	0	0	2,5
170	0	0	0	0	0,000122	0	0	0	2,5
180,01	0,01	0	0	0	0,000078	0	0	0	2,5
190	0	0	0	0	0,000034	0	0	0	2,5

Bitte beachten Sie:

- Die erste Zeile ist der sog. Header. Er enthält die Namen aller Werte, die aufgezeichnet wurden und zwar so, wie wir es im `Record = { ... }`; Abschnitt aus dem letzten Kapitel festgelegt haben.
- Die erste Spalte A wird von SILAB automatisch hinzugefügt. Sie enthält in jeder Zeile den Messzeitpunkt (gemessen in Millisekunden seit dem Start der Simulation), zu dem die Werte in den restlichen Spalten geschrieben wurden. Die Messzeitpunkte werden in regelmäßigen Schritten erhöht, im Beispiel jeweils um 10 ms. Das entspricht einer Aufzeichnungsfrequenz von 100 Hz. Mit dieser Frequenz führt der Rechner, auf dem die `DPUDataFile` läuft, Simulationsschritte durch. Je nach Konfiguration und Auslegung Ihres Simulators können hier auch andere Zeitschritte stehen, z.B. 8.3333 ms. Das bedeutet, dass der Aufzeichnungsrechner mit einer Frequenz von 120 Hz arbeitet.
- Auch die zweite Spalte B fügt SILAB automatisch ein. Sie enthält den sog. Messzeitverzug. Wenn der Aufzeichnungsrechner stark überlastet ist (z.B. durch ein zu komplexes Szenario mit zu vielen simulierten Verkehrsteilnehmern), dann gerät er bei der Berechnung seiner DPUs in Verzug. Wie hoch der Verzug (in Millisekunden) beim aktuellen Messzeitpunkt war, sehen Sie in dieser Spalte B. Idealerweise hat der Aufzeichnungsrechner natürlich keine Verzögerungen, das heißt in Spalte B findet sich nur der Wert 0. Kleine Verzögerungen wie in unserem Beispiel sind jedoch tolerierbar. Hauptsache, die Spalte enthält dann trotzdem in der überwiegenden Zahl der Fälle den Wert 0 ms.
- Ab Spalte C finden Sie dann die Werte zu den Signalen, die wir im `Record = { ... }`; Abschnitt im vorhergehenden Kapitel festgelegt haben.

Grob gesprochen besteht die Auswertung von Fahrsimulationsdatei aus drei Schritten:

1. Segmentierung der Daten
2. Berechnung abgeleiteter Maße
3. Berechnung von Kennzahlen und Interpretation der Ergebnisse

Wir wollen diese Schritte an unserem Beispiel nachvollziehen.

4.10.1 Segmentierung der Daten

In unsere Autobahnstrecke haben wir einen Streckenabschnitt `start` eingefügt, auf dem der Fahrer Zeit hat, ein stabiles Folgefahren herzustellen. Für die Auswertung interessiert uns dieser Abschnitt nicht. Wir wollen nur das eigentliche Folgefahren im Streckenabschnitt `road` analysieren.

Öffnen Sie die Datei „tutorial_step_09.cfg“ in Ihrem Texteditor und suchen Sie die Stelle `### courses`. In der Definition des Start-Abschnittes (`define Course Start { ... };`) finden Sie die Zeile

```
NodeID = 0;
```

Auch die Definition des Streckenabschnittes für das eigentliche Folgefahren (`define Course HighwayRoad { ... };`) enthält eine solche Zeile:

```
NodeID = 1;
```

Es handelt sich dabei um Kennungen für die Streckenabschnitte. Diese Kennungen können Sie frei vergeben. Sie werden in die entsprechende Spalte H der Aufzeichnungsdatei geschrieben, wenn sich der Fahrer gerade auf dem jeweiligen Streckenabschnitt befindet. Markieren Sie in Excel alle Zeilen mit dem Wert 0 in Spalte H. Das ist der gesamte Zeitraum, in dem sich der Fahrer auf dem Start-Abschnitt befunden hat. Löschen Sie die markierten Zeilen. Die Aufzeichnungsdatei enthält jetzt nur noch die Daten des interessierenden Folgefahrt-Abschnitts.

4.10.2 Berechnung abgeleiteter Maße

Wir möchten die Fahrleistung anhand von zwei gebräuchlichen Maßen beurteilen:

Zeitabstand: Dieses Maß gibt an, wie lange es bei der momentanen Geschwindigkeit dauert, um eine Strecke zurückzulegen, die dem momentanen Abstand zum Vordermann entspricht. Beispielsweise entspricht ein Zeitabstand von 2 s zum Vorderfahrzeug bei einer Geschwindigkeit von 100 km/h einer Entfernung von ca. 55,6 m.

Querabweichung: Dieses Maß erfasst die Güte der Spurhaltung. Gemessen wird in jedem Zeitschritt, welchen Abstand der Fahrzeugschwerpunkt von der Ideal-Linie (d.h. der exakten Mitte) der Fahrbahn hat.

Während die Querabweichung bereits fertig in unserer Aufzeichnungsdatei enthalten ist (Spalte K, „LateralDistance“), müssen wir den Zeitabstand erst aus den vorhande-

nen Messwerten berechnen. Hierzu beschriften wir die zusätzliche Spalte N im Header mit „TimeDistance“ und fügen in Zeile 2 folgende Formel ein:

SUMME												
	D	E	F	G	H	I	J	K	L	M	N	O
1	BrakePedal	v_kmh	ax	ay	NodeTemplateID	s	LaneWidth	LateralDistar	d	v	TimeDistance	
2	0	84,179771	-0,384871	0	1	0,093339	3,75	0,074457	24,088391	22,22	=L2/(E2/3,6)	
3	0	84,165916	-0,38483	0	1	0,327172	3,75	0,074395	24,076758	22,22		
4	0	84,152061	-0,384788	0	1	0,560966	3,75	0,074332	24,065164	22,22		
5	0	84,138214	-0,384746	0	1	0,794722	3,75	0,07427	24,053608	22,22		

Erweitern Sie die Formel auf alle Zeilen von Spalte N.

4.10.3 Berechnung von Kennzahlen und Interpretation der Ergebnisse

Als Kennzahl für die Qualität der lateralen Fahrzeugführung über die ganze Fahrt hinweg verwenden wir die Standardabweichung der Spurabweichung (SDLP, „standard deviation of lane position“). Die Qualität der longitudinalen Fahrzeugführung beurteilen wir anhand der Standardabweichung des Zeitabstands.

Fügen Sie am Anfang der Excel-Tabelle zwei neue Zeilen ein. Beschriften Sie die Zeilen in Feld A1 mit „SDLP“ und in Feld A2 mit „SD(TimeDistance)“. Fügen Sie in B1 die Formel für die Standardabweichung aller Werte in Spalte K ein:

=STABW.N(K4:K22428)

Bitte beachten Sie: Je nachdem, wie viele Werte Ihre Aufzeichnungsdatei enthält, variiert auch die obere Bereichsgrenze in den Klammern.

Auch in Feld B2 wird die entsprechende Standardabweichung berechnet:

=STABW.N(N4:N22428)

Achtung: Wenn Sie mit der englischen Version von Excel arbeiten, dann heißt die Formel für die Standardabweichung „STDEVP(...)“.

Ihre Excel-Tabelle sollt jetzt etwa so aussehen:

B1												
	A	B	C	D	E	F	G	H	I	J	K	
1	SDLP	0,273207555										
2	SD(TimeDistance)	0,318233108										
3	MeasurementTime	Measurement time error	AcceleratorPedal	BrakePedal	v_kmh	ax	ay	NodeTemplateID	s	LaneWidth	LateralC	
4	105330	0	0	0	84,179771	-0,384871	0	1	0,093339	3,75	0,07	
5	105340	0	0	0	84,165916	-0,38483	0	1	0,327172	3,75	0,07	
6	105350,01	0,01	0	0	84,152061	-0,384788	0	1	0,560966	3,75	0,07	
7	105360,01	0,01	0	0	84,138214	-0,384746	0	1	0,794722	3,75	0,0	
8	105370,01	0,01	0	0	84,124352	-0,384704	0	1	1,028439	3,75	0,07	
9	105380,02	0,02	0	0	84,110512	-0,384662	0	1	1,262118	3,75	0,07	
10	105390,01	0,01	0	0	84,096664	-0,384621	0	1	1,495758	3,75	0,07	
11	105400,01	0,01	0	0	84,082809	-0,384579	0	1	1,72936	3,75	0,0	
12	105410	0	0	0	84,06897	-0,384537	0	1	1,962923	3,75	0,07	
13	105420,01	0,01	0	0	84,05513	-0,384495	0	1	2,196448	3,75	0,07	
14	105430,01	0,01	0	0	84,041283	-0,384453	0	1	2,429935	3,75	0,07	
15	105440	0	0	0	84,027443	-0,384412	0	1	2,663383	3,75	0,07	
16	105450	0	0	0	84,013603	-0,38437	0	1	2,896792	3,75	0,07	

Die Interpretation der Ergebnisse wäre bei einer beispielhaften Fragestellung folgende: Angenommen, Sie haben unser Szenario benutzt, um zwei Auslegungen A und B eines Bedienkonzepts für ein Informationssystem zu beurteilen. Dafür haben Sie $N=20$ Fahrer instruiert, die Strecke mit möglichst guter Spurhaltung und konstantem Abstand zum Vorderfahrzeug zu durchfahren. Eine Fahrergruppe bedient dabei das Informationssystem mit Auslegung A, die andere mit Auslegung B. Wenn Sie nach der in diesem Schritt des Tutorials skizzierten Auswertung feststellen, dass die Fahrer mit Variante A im Schnitt eine höhere SDLP und eine höhere SD(TimeDistance) haben als die mit Variante B, dann haben Sie gute Argumente, dass Auslegung B die Fahrleistung weniger beeinträchtigt als Auslegung A. Die Signifikanz dieser Aussage sollten Sie natürlich noch überprüfen.

Das an dieser Stelle skizzierte Vorgehen dient vor allem der Veranschaulichung einer praktischen Umsetzung. Genaue Testanweisungen für die Durchführung eines standardisierten Testprotokolls zur Prüfung von Ablenkung (z.B. Anforderungen Probanden, Testaufbau, Durchführung und Auswertung) finden sich in AAM (2006) oder NHTSA (2012).

4.11 Schritt 11: Auffahrt aus Parkplatz (SILABAEEdit)

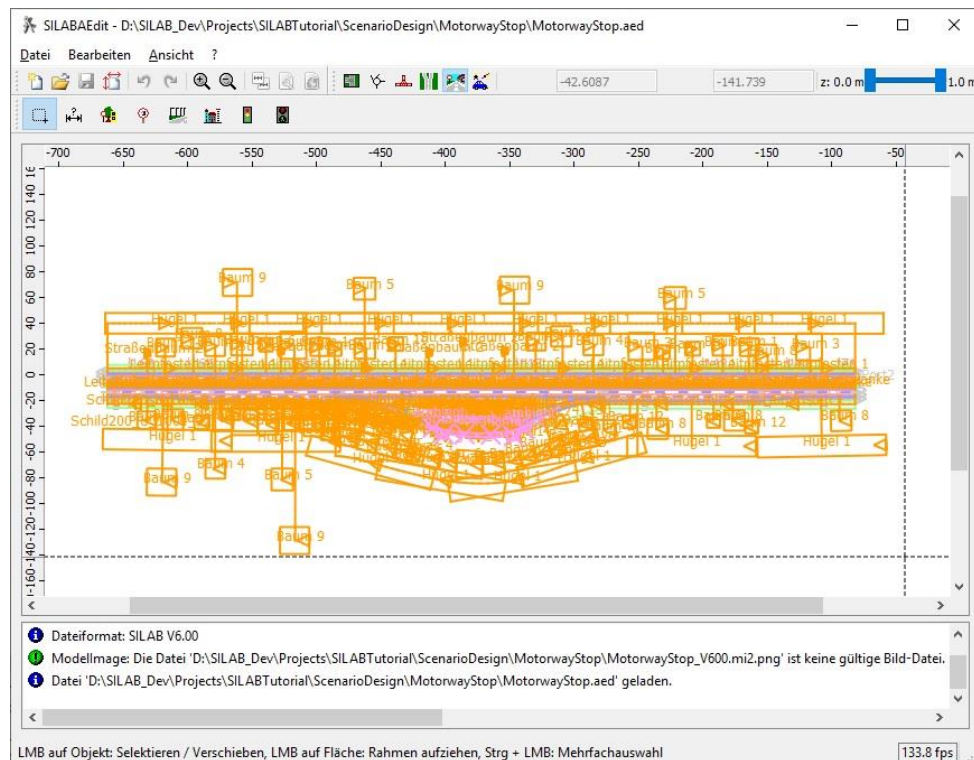
Zu Beginn der Simulation ist unser Szenario etwas unrealistisch: Der Fahrer steht irgendwo auf dem Standstreifen einer Autobahn und vor ihm (und noch dazu auf einer Fahrspur) steht ein anderes Fahrzeug. In diesem Schritt des Tutorials gestalten wir die Start-Situation realistischer.

In den Schritten 1 – 4 haben wir die Autobahn als Straßenstücke vom Typ Course modelliert. Mit diesem Typ können Sie ganz allgemein Straßen mit durchgängig gleichem Querschnittsprofil und ohne Abzweigungen modellieren. Alle anderen Streckenstücke, also z.B. Fahrbahnerweiterungen/-verengungen, Kreuzungen, Autobahnauf- und abfahrten sowie ganze Stadtteile werden in SILAB als sog. Area2 entworfen.

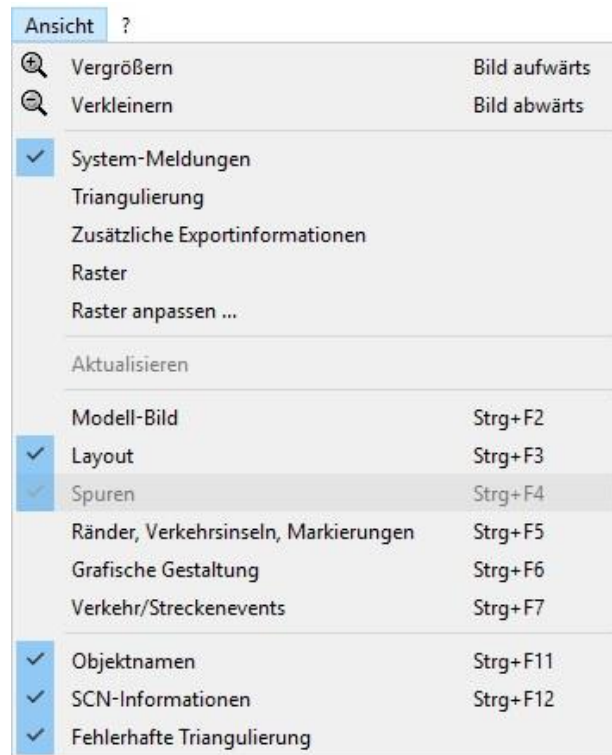
Wir möchten unsere Strecke an einem Rastplatz beginnen lassen. Das vorausfahrende Fahrzeug soll ebenfalls auf dem Rastplatz warten und mit dem Fahrer auf die Autobahn fahren. Ein Rastplatz besteht aus verschiedenen Querschnittsprofilen (nämlich das der vorbeiführenden Autobahn und die Querschnittsprofile der Straße durch den Rastplatz) und aus einer Ab- und Auffahrt von bzw. zur Autobahn. Wir modellieren den Rastplatz also als Area2. Da Area2 Streckenstücke sehr komplexe Spurgeometrien haben können (denken Sie an eine komplizierte X-Kreuzung in der Stadt), werden sie nicht im Skript entworfen. Es gibt dafür einen komfortablen graphischen Editor namens „SILABAEEdit“, in dem Sie die Area2 „zeichnen“.

- Starten Sie diesen Editor. Entweder es gibt bereits eine Verknüpfung auf Ihrem Desktop oder Sie starten das Programm direkt aus dem Unterverzeichnis „bin“ Ihres SILAB Installationsverzeichnis.
- Wählen Sie *Datei* → *Öffnen* und laden Sie die Datei „MotorwayStop\MotorwayStop.aed“ aus dem Tutorialordner.
- Bewegen Sie die Maus in den Zeichenbereich, so dass Sie das Fadenkreuz erkennen können. Verschieben (mit den Scroll-Leisten des Zeichenbereichs)

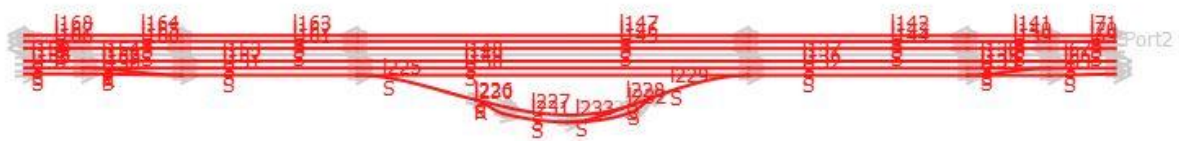
und Zoomen Sie die Zeichnung (mit dem Mausrad), bis Sie den ganzen Parkplatz sehen können. Ihre Darstellung sollte jetzt etwa wie folgt aussehen:



Aufgrund der vielen Informationen, die gleichzeitig dargestellt werden, ist kaum etwas zu erkennen. Wir wollen uns Schrittweise die einzelnen „Informationsebenen“ ansehen und beginnen mit den Querschnitten und Spuren. Wählen Sie *Bearbeiten* → *Spuren*. Stellen Sie sicher, dass im Menüpunkt *Ansicht* ab dem Menüpunkt *Modell-Bild* nur Häkchen an den Einträgen *Querschnitte* und *Spuren* sind.

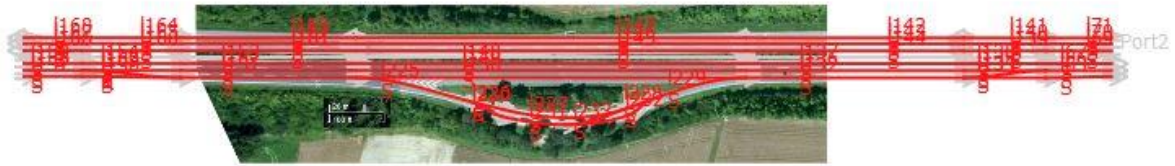


Die Zeichnung sollte jetzt so aussehen:



Sie erkennen die Autobahnspuren, die im oberen Bereich horizontal von links nach rechts verlaufen, und in der Mitte die abzweigende Spur, die durch den Parkplatz wieder auf die Autobahn führt. Parallel zu dieser Spur verläuft im Parkplatz die eigentliche Parkspur. Zoomen Sie mit dem Mausrad an verschiedenen Stellen. Schauen Sie sich die schwarz dargestellten Querschnittsprofile mit ihren Pfeilen für die Fahrrichtungen der Spuren an. SILABAEEdit bietet Zeichentools, mit denen Sie solche Querschnittsprofile und die Spuren dazwischen entwerfen können.

Die Geometrie des Parkplatzes ist einem real existierenden Parkplatz nachempfunden. Hierzu bietet SILBAEdit die Möglichkeit, ein Luftbild oder eine Konstruktionszeichnung als eine Art Hintergrundbild einzubetten. Wenn Sie *Ansicht* → *Modell-Bild* wählen, sehen Sie die Vorlage für diesen Parkplatz:



Deaktivieren Sie *Ansicht* → *Modell-Bild* und blenden Sie mit *Ansicht* → *Ränder, Verkehrsinseln, Markierungen* eine weitere Informationsebene ein. Untersuchen Sie (durch Mausrad und Scrollen) verschiedene Stellen. Sie erkennen, dass mit dieser Ebene Fahrbahnmarkierungen (blau) definiert werden. Zudem werden Bereiche neben der Straße als Ränder markiert. Grün gezeichnete Ränder werden in der Simulation als Grasflächen dargestellt, grau gezeichnete als erhöhte Pflasterfläche (Gehsteig).

Blenden Sie die nächste Ebene ein (*Ansicht* → *Grafische Gestaltung*). Zoomen Sie auf den eigentlichen Parkplatz:

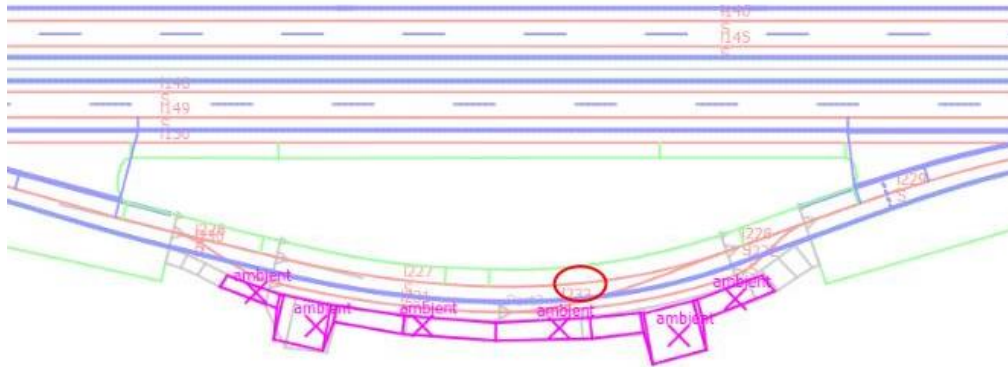


Sie erkennen die grafischen Objekte, die für die Ausgestaltung eingefügt wurden. Beispielsweise gibt es links unten einen Bereich mit Bänken und einem Mülleimer. Auch die parkenden Autos („Fiat 500“, „Hyundai i40“ usw.) sind keine simulierten Verkehrsteilnehmer sondern rein grafische Objekte. An anderen Stellen der Zeichnung finden Sie Leitpfosten, Bäume, Schilder und Hügel. Blenden Sie der Übersichtlichkeit halber diese Ebene wieder aus.

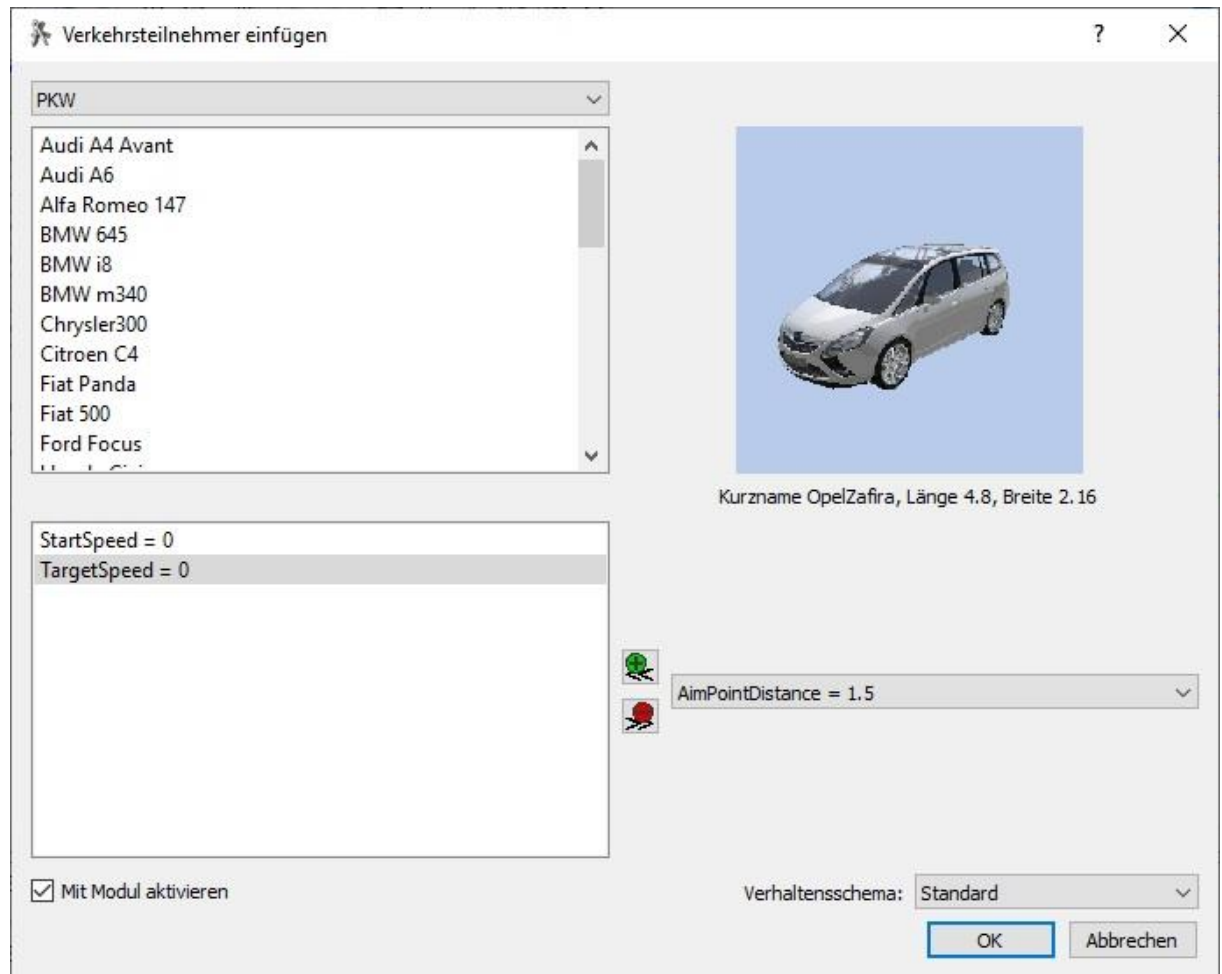
Aktivieren Sie die nächste Ebene (*Ansicht* → *Verkehr/Streckenevents*). In dieser Ebene werden die simulierten Verkehrsteilnehmer definiert. Momentan sehen Sie im Parkplatzbereich nur pink dargestellte Flächen, die auf dem Gehweg liegen. Es handelt sich dabei um Bereiche, in denen sich selbständig Fußgänger bewegen. Es gibt noch keine Fahrzeuge (außer natürlich die rein grafischen parkenden).

Wir werden jetzt unser vorausfahrendes Fahrzeug definieren.

- Blenden Sie der Übersichtlichkeit halber die Ebene *Ränder*, *Verkehrsinself*, *Markierungen* aus.
- Wählen Sie *Bearbeiten* → *Verkehr/Streckenevents*.
- Selektieren Sie das Tool *Verkehrsteilnehmer aufsetzen* aus der Toolbar (🚗).
- Klicken Sie auf die im folgenden Bild mit einem blauen Kreis markierte Spur:

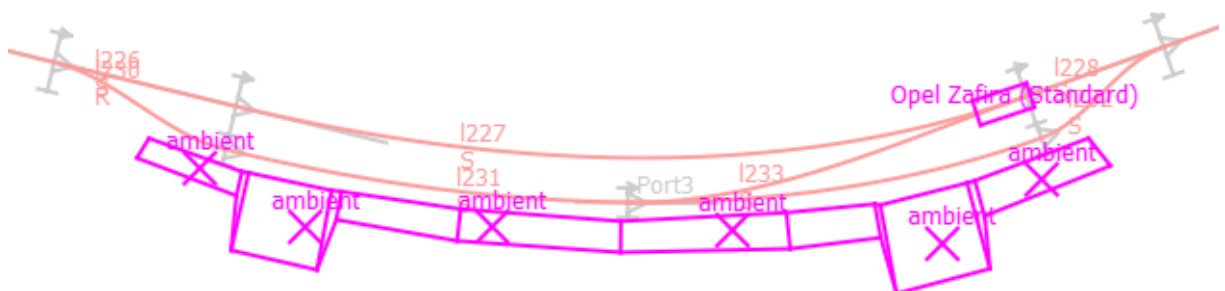


- Es erscheint der Dialog zum Einfügen eines Verkehrsteilnehmers. Nehmen Sie folgende Einstellungen vor:



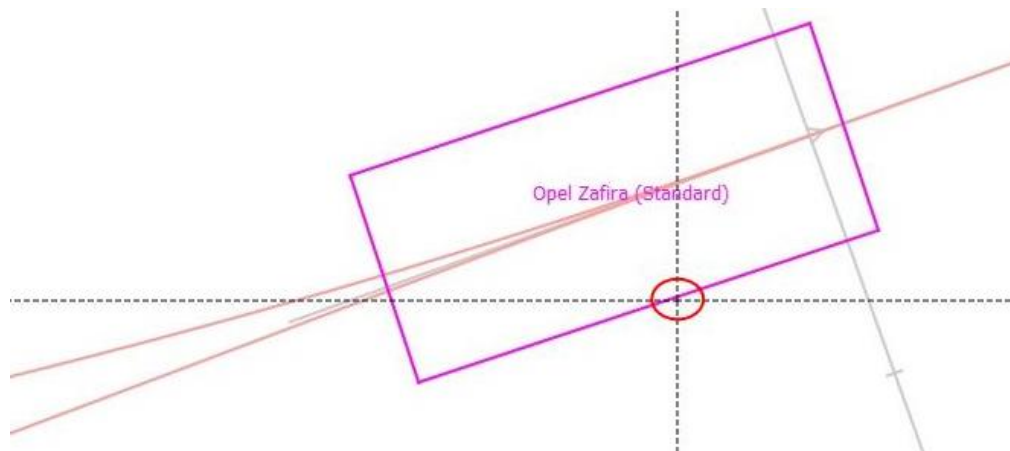
Sie haben die Einstellung bestimmt wiedererkannt: Es sind die gleichen, die wir in Schritt 6 für das vorausfahrende Fahrzeug im Skript festgelegt haben. Beenden Sie den Dialog durch Klick auf OK.

- Verschieben Sie das Fahrzeug auf der Spur, bis es an der im folgenden Bild dargestellten Position ist. Klicken Sie dann mit der linken Maustaste.



- Ausgelöst durch das Losfahren des Fahrers soll der Vorausfahrende auch starten und langsam auf die Zielgeschwindigkeit 80 km/h beschleunigen. Wählen Sie aus der Toolbar das Tool *Flowpoint einfügen* (🚦). Zoomen Sie

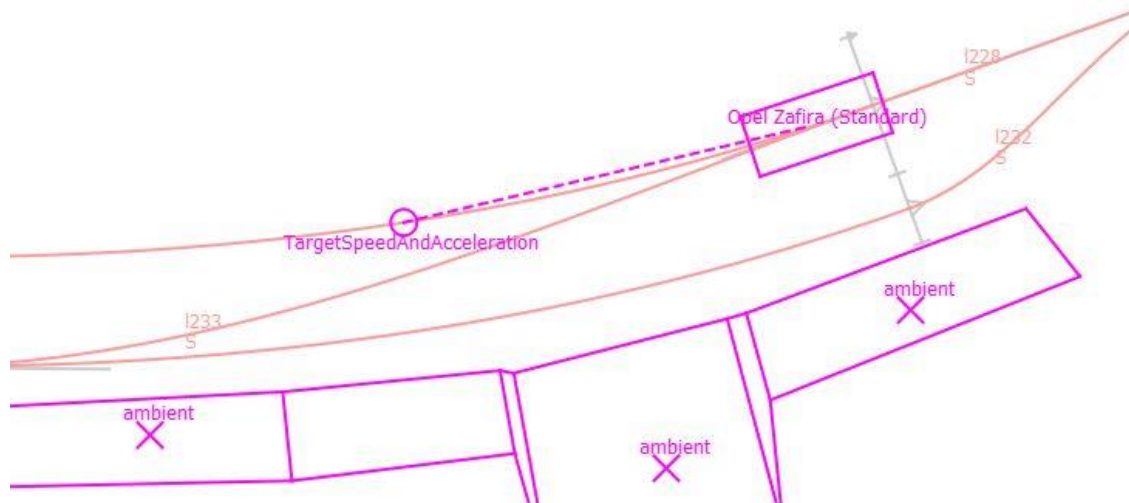
nah an das vorausfahrende Fahrzeug heran und klicken Sie auf den Rand seines Rechtecks (blauer Kreis):



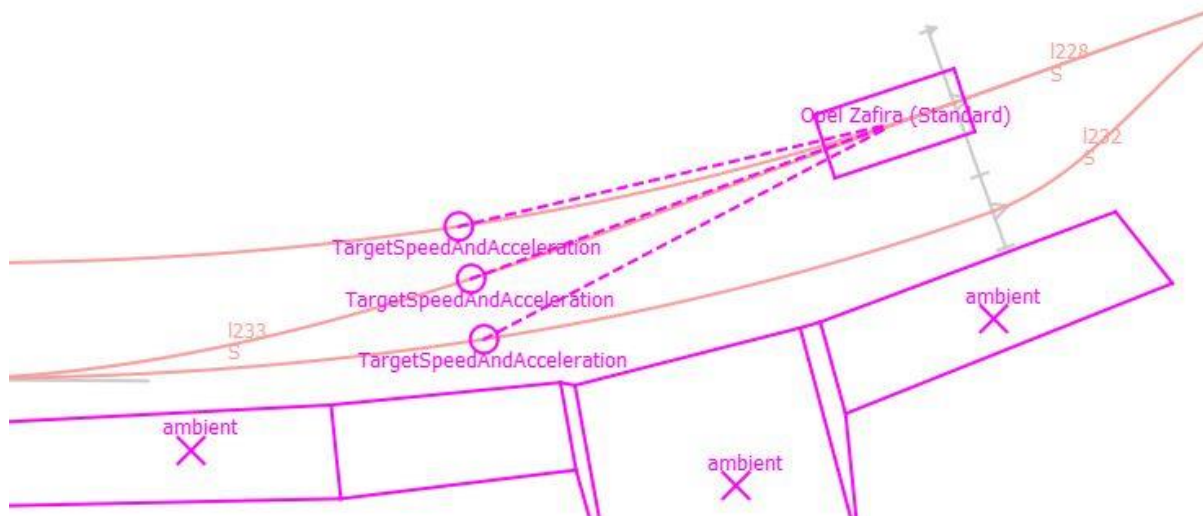
- Es erscheint folgender Dialog, in dem Sie die gezeigten Einstellungen vornehmen:

Das Bild zeigt ein Software-Fenster mit dem Titel 'Flowpoint einfügen'. Oben befindet sich eine Dropdown-Liste mit der Auswahl 'TargetSpeedAndAcceleration'. Darunter steht die Beschreibung: 'Setzt die Zielgeschwindigkeit, Beschleunigung und Verzögerung des Fahrzeugs.' Im Zentrum des Fensters ist ein Textfeld mit den folgenden Werten: 'acceleration (m/s²) = 1', 'deceleration (m/s²) = -2.5' und 'v (m/s) = 22.22'. Unten links ist ein Kontrollkästchen mit der Aufschrift 'für EGO-Fahrzeug' aktiviert. Darunter befindet sich eine weitere Dropdown-Liste mit der Auswahl 'SimCar'. Rechts neben dieser Liste sind zwei leere Textfelder. Am unteren Rand des Fensters befinden sich zwei Buttons: 'OK' und 'Abbrechen'.

- Beenden Sie den Dialog und klicken Sie auf die Spur, an welcher der Flowpoint positioniert werden soll (s. Bild). Verschieben Sie den Flowpoint anschließend auf dieser Spur an die gewünschte Stelle wie im folgenden Bild und klicken Sie:



- Da wir nicht wissen, auf welchen Spuren der Fahrer aus dem Parkplatz fahren wird, müssen wir auf allen drei möglichen Spuren den gleichen Flowpoint platzieren. Führen Sie dazu die vorhergehenden Schritte noch zweimal aus, so dass Sie folgendes erhalten:



- Wählen Sie *Datei* → *Exportieren* → *SILAB* und bestätigen Sie die Voreinstellungen mit „OK“.

Mit dem letzten Schritt haben Sie den Rastplatz in einem Format exportiert, in dem er in unser Szenario integriert werden kann. Das machen wir jetzt.

- Öffnen Sie „tutorial_step_11.cfg“.
- Suchen Sie die mit `### Area2: motorway stop` markierte Stelle:

```
### Area2: motorway stop
include MotorwayStop/MotorwayStop_Area2.cfg
### /Area2: motorway stop
```

Auf der gleichen Ebene, in der wir unsere Course-Stücke (mit `define Course Start { ... };` und `define Course HighwayRoad { ... };`) definiert haben, wird mittels `include` die Datei „MotorwayStop_Area2.cfg“ eingebunden. Bei dieser Datei handelt es sich um ein Skript, das SILABAEit beim Export aus unserer Zeichnung erzeugt hat. Wenn Sie diese Datei in Ihrem Texteditor öffnen, dann finden Sie folgende Struktur:

```
define Area2 MotorwayStop
{
    (der Übersichtlichkeit halber weggelassen)
};
```

Völlig analog zu unseren Course-Stücken wird hier ein Area2 Streckenstück definiert.

- Suchen Sie in der Datei „tutorial_step_11.cfg“ die mit `### road network` markierte Stelle.

```
### road network
MotorwayStop motorwayStop;
Start start;
HighwayRoad road;
Connections =
{
    motorwayStop.Port2 <-> start.Begin,
    start.End <-> road.Begin,

    motorwayStop.Port3 <-> Port1,
    road.End <-> Port2
};
### /road network
```

Zunächst wird – wieder genau wie bei den Course Streckenstücken – eine Instanz `motorwayStop` des Area2-Typs `MotorwayStop` angelegt. Mit dieser Instanz wird das Streckennetz im Abschnitt `Connections = { ... };` erweitert, indem sie vor die Instanz `start` gehängt wird.

- Suchen Sie die Markierung `### traffic`

```
### traffic
# include leading vehicle from motorway stop
include MotorwayStop/MotorwayStop_TRF.cfg
Traffic MyTraffic
{
    ### overtaking traffic
    TrafficFlow OvertakingTraffic
    {
        (der Übersichtlichkeit halber weggelassen)
    };
    ### /overtaking trafic

    ### oncoming traffic
    TrafficFlow OncomingTraffic1
    {
        (der Übersichtlichkeit halber weggelassen)
    };

    TrafficFlow OncomingTraffic2
    {
        (der Übersichtlichkeit halber weggelassen)
    };
    ### /oncoming traffic
};
### /traffic
```

Die Steuerung des vorausfahrenden Fahrzeugs auf dem Parkplatz hat SILABAEEdit in der Datei „MotorwayStop_TRF.cfg“ exportiert. Diese Datei wird gleich zu Anfang eingebunden. Der folgende Abschnitt entspricht der Verkehrsdefinition, die wir in den Schritten 7 und 8 dieses Tutorials erstellt haben. Der Definition des Vorausfahrenden auf dem Course Streckenstück `start` aus Schritt 6 wurde entfernt.

Laden Sie „tutorial_step_11.cfg“ in SILAB, machen Sie eine kurze Pause auf dem Rastplatz und folgen Sie dann dem weißen Opel Zafira auf die Versuchsstrecke.



5 WIE GEHT ES WEITER?

Das Szenario, das wir in diesem Tutorial entworfen haben, ist relativ einfach. Im Laufe Ihrer Arbeit mit SILAB werden Sie deutlich komplexere Szenarien entwickeln, in denen Sie die zahlreichen Features von SILAB noch intensiver nutzen werden.

Die im Lieferumfang von SILAB enthaltene Dokumentation hilft Ihnen dabei, die Möglichkeiten kennen zu lernen und die zugrunde liegenden Mechanismen richtig einzusetzen. Die folgende Tabelle soll Ihnen einen Überblick über die Dokumentation geben. Die Reihenfolge, in der die Dokumente aufgeführt sind, ist auch ein Vorschlag für die Lesereihenfolge.

Dokument	Inhalt
SILAB Gesundheits- und Sicherheitshinweise	Enthält Informationen zum Umgang mit dem Simulator sowie zum Umgang mit der sog. Simulator Sickness.
Handbuch SILAB Essential	Befasst sich mit allen Aspekten, die die Bedienung von SILAB zur Durchführung einer Simulation betreffen, d.h. die Verwaltung des Rechnernetzes Ihres Simulators, die Benutzeroberfläche von SILAB und die Konstruktion von Versuchsstrecken aus von uns erstellten fertigen Szenarienpaketen.
Handbuch SILAB Professional	Hier werden die verfügbaren Möglichkeiten zur Gestaltung Ihrer eigenen Strecken in größerer Detailtiefe vorgestellt.

Referenz Datenbasis SCNX	Beschreibt im Detail alle Möglichkeiten, die SILAB zum Entwurf von Szenarien bietet. Die Definition der Verkehrssimulation sowie die Bedienung von SILABAEEdit finden Sie in gesonderten Dokumenten (s.u.). Das Dokument ist zum „Nachschlagen“ gedacht. Es lohnt sich aber in jedem Fall, den Inhalt im Überblick zu kennen.
Referenz Verkehrssimulation TRFX	Beschreibt im Detail alle Möglichkeiten, simulierten Verkehr für Szenarien zu definieren. Auch in diesem Dokument werden Sie während Ihrer Arbeit mit SILAB viel nachschlagen. Versuchen Sie trotzdem, einen Überblick seines Inhalts zu bekommen.
Referenz Fußgängersimulation MOV	Beschreibt die Möglichkeiten der Fußgängersimulation. Auch hier lohnt es sich, zumindest einen Überblick zu haben.
Handbuch SILABAEEdit	Detaillierte Bedienungsanleitung zum Programm SILABAEEdit, mit dem Sie Area2 Streckenstücke (wie unser Parkplatz aus Abschnitt 4.11) entwerfen.
Handbuch Szenariogestaltung	Eine Sammlung von Ideen und Tipps, die Sie in Ihren Szenarien verwenden können. Beispielsweise erfahren Sie, wie Sie gezielt das Wetter beeinflussen können und wie Sie Tunnel und Brücken realisieren.
Referenz SILAB-Konfiguration und Bedienung	Detaillierte Beschreibung der grundlegenden Funktionsweise von SILAB. Sie erfahren u.a. mehr über DPUs und deren Verschaltung, weiteren Möglichkeiten der Benutzeroberfläche von SILAB, der Fernsteuerung von SILAB sowie zu dem Tool zur Verwaltung Ihrer Simulator-Rechner. Insbesondere die Teile über die DPUs und die Standard-Konfigurationsarchitektur sollten Sie lesen, wenn Sie die Funktionalität von SILAB erweitern möchten (z.B. um externe Hard-/Software oder Funktionalitäten aus dem Lieferumfang von SILAB zu integrieren).
DPU Referenzen	Im Unterverzeichnis „DPU“ des Dokumentationsverzeichnisses von SILAB finden Sie detaillierte Beschreibungen aller DPUs, die im Lieferumfang von SILAB enthalten sind. Beispielsweise gibt es ein Dokument „DPUDataFile.pdf“, in dem die Funktionsweise der Aufzeichnungs-DPU aus Abschnitt 4.9 erläutert wird.
Handbuch SILAB Enterprise	Lesen Sie dieses Dokument, wenn Ihre SILAB-

	Lizenz die Enterprise-Edition beinhaltet und Sie eigene DPUs (z.B. virtuelle Sensoren, Fahrerassistenzsysteme, Nebenaufgaben für den Fahrer) in C++/ JAVA programmieren oder mit Matlab/Simulink entwickeln möchten.
Referenz Visualisierung SGE	Mit diesem Dokument arbeiten Sie, wenn Sie die grafische Darstellung von SILAB anpassen oder erweitern möchten. Es enthält u.a. Beschreibungen, wie Sie eigene grafische Objekte integrieren können und Screenshots oder digitale Videos von Fahrszenen aufnehmen können.

Darüber hinaus helfen wir Ihnen natürlich auch persönlich. Wenn Sie Probleme mit einem Feature von SILAB haben, eine Stelle der Dokumentation unklar ist oder wenn Sie einfach nur wissen möchten, wie Sie bei der Konfiguration von SILAB oder der Umsetzung eines Szenarios für Ihre Fragestellung vorgehen sollen, dann wenden Sie sich bitte jeder Zeit an unseren Support.

6 REFERENZEN

National Highway Traffic Safety Administration (2012). Visual Manual NHTSA Driver Distraction Guidelines for In-Vehicle Electronic Devices. Washington, DC: National Highway Traffic Safety Administration (NHTSA), Department of Transportation (DOT).

Alliance of Automobile Manufacturers (2006). Statement of Principles, Criteria and Verification Procedures on Driver Interactions with Advanced In-Vehicle Information and Communication Systems. Washington, D.C.: Alliance of Automobile Manufacturers (AAM).